CMPE4003 – CPU and GPU Architectures

Basic 16-bit CPU Implementation Report

Chihab Chamkhi	Xavier Curr	Vishaal Chopra	Afeef Ahmed
21498754	22199456	20718930	20051376

October 2026

The video explanation of the design is accessible here `https://youtu.be/KxaZ6iwYb2Yhttps://youtu.be/KxaZ6iwYb2Y`
The Code and commit history can be found at this link `https://github.com/Nihlitect/basicCPU`

Contents

1 Introduction **5**

2 Design Specifications and Architecture Overview **5**
2.1 Memory Structures . . . 5
2.2 Instruction Set Architecture . . . 6
2.2.1 Instruction Types . . . 6
2.3 From 8-bits to 16-bits . . . 7
2.4 Functions . . . 8
2.4.1 ALU and ALUI Functions . . . 8
2.4.2 ALU2 and ALU2I Functions . . . 8
2.4.3 SHIFT Functions . . . 8
2.4.4 BR Functions . . . 9
2.4.5 IO Functions . . . 9

3 Component Design and Description **9**
3.1 Program Counter (PC) . . . 9
3.2 Memory (ROM and RAM) . . . 11
3.2.1 Instruction Memory (ROM) . . . 11
3.2.2 Data Memory (RAM) . . . 12
3.3 Registers . . . 12
3.4 Control Unit . . . 13
3.5 Branch Unit . . . 15
3.6 Forward Select Unit . . . 16
3.7 Memory Hazards . . . 18
3.8 IO (SSD and Switches) . . . 19
3.9 ALU1, ALU2 and Shift Unit (VHDL) . . . 20
3.10 ALU1 . . . 20
3.11 ALU2 . . . 22
3.12 SHIFT Unit . . . 23
3.13 Register, Pipeline Register and Flag Latch . . . 24
3.13.1 Register File . . . 24
3.13.2 Pipeline Registers . . . 24
3.13.3 Flag Latch . . . 25
3.14 Supporting Components . . . 25
3.14.1 Demultiplexer . . . 25
3.14.2 Multiplexers, Buses . . . 25

4 Top-Level Design and Data Flow **26**
4.1 Top-level Pipelined Architecture . . . 26
4.1.1 Fetch Stage . . . 26
4.1.2 Decode Stage . . . 26
4.1.3 Execute Stage . . . 27
4.1.4 Writeback Stage . . . 30
4.2 Worked example walkthrough . . . 30
4.2.1 I-Type Load Instructions . . . 30
4.2.2 R-Type ALU Instructions . . . 33
4.2.3 J-Type Program Flow Instructions . . . 33
4.3 Clock cycles per instruction . . . 36

5 Testing and Verification **36**
5.1 Component Test benches 36
5.2 On-Board Debugging 37
5.3 Self-Checking Test Programs 38
5.4 Results 38

6 Discussion **39**

7 Conclusion **40**

A Assembly of Code ran on the CPU **40**
A.1 Assembly code for 'echo' program 40
A.2 Assembly Code for Calc Program 41

B Testing. Decoding Tables **43**

C Assembler **44**
C.1 Code Examples 44

List of Tables

1 Data structures used in the CPU . . . 5
2 Instruction Set Architecture used in CPU design. . . . 6
3 Instruction types and their bit formats used. . . . 7
4 ALU func codes and respective functions . . . 8
5 Table of the functions for ALU2. . . . 8
6 Table of the four shift functions, that take 4-bit amount(0-15) . . . 9
7 Funciton Select in branch instrucitons for various conditional jumps. . . . 9
8 Table of IO Functions. There is a lot more room for development and functionality . . . 9
9 Description of functionality of the various input signals of Program Counter . . . 10
10 Descriptions of the various output signals of Program Counter . . . 11
11 Pipeline registers used in the top-level design. . . . 25
12 Worked example Tabular summary through clock cycles . . . 32
13 Clock Cycles per instruction type. . . . 36
14 VHDL testbenches used to verify individual components. . . . 37
15 Results of the test programs on the final design. . . . 39
16 Some Faults found during testing and their fixes . . . 39
17 Code used to decode the funnel output in execute stage of the pipeline. . . . 43
18 Table used to decode CPU functionality with LEDs. . . . 43
19 Table for selecting output (with push button switches) onto seven segment display 5 and 6. . 44

List of Figures

1 Block Diagram form of the program counter Implementation. . . . 10
2 Rom Implementation for Instruction memory . . . 11
3 Random Access Memory for Program Data storage, similar to a heap . . . 12
4 VHDL entity definition of dual-port Registers . . . 12
5 Jump and branch functional block implementation in CPU. . . . 16
6 Block Diagram Implementation of Forward select Unit in the CPU. . . . 18
7 Block Diagram showing a Register connected to the pinout for the seven segment displays. Register is connected to the shared bus. A mux is used to tap into the seven segment display output for on-device-debugging purposes. . . . 20
8 Block Diagram of funnel network for execute output stage and interaction with the FPGA Switch inputs. Ofcourse device technically only has 12 switches the 4 gnd signals are used to make it 16-bit. . . . 21
9 Top Level view of CPU design. View of Stages and Instruction Flow. . . . 26
10 Block Diagram implementation of the fetch stage, showing PC integration. . . . 27
11 Close up block Diagram of Decode stage of the pipeline with all functional component interactions 28
12 Close up block diagram implementation of Execute Stage of Basic CPU, . . . 29
13 Close up block diagram of Write back Stage. . . . 31
14 The PC addresses the ROM at 0. On the rising clock edge, the instruction is fetched and passed to the Decode stage. As the instruction is 32 bits long, the PC is set to its current value +2. . . . 32
15 An `LDI` instruction in the Execute stage simply routes its immediate value to the Execute output. . . . 33
16 The `ADD` instruction needs the values of registers that have not yet been written back, so they are forwarded into the Decode stage. . . . 34
17 `SSD R2` in Decode receives the result of `ADD` forwarded from the Execute stage. . . . 34
18 `BEQ 1` is fetched while `CMP` is in Decode. . . . 35
19 `BEQ` has executed and the branch is taken, setting the PC to 1. . . . 35
20 Block Diagram of Status Indicators on FPGA used for debugging purposes. . . . 37
21 Program dependencies and debugging flow used in cpu desing. . . . 38

1 Introduction

Modern processors rely on pipelining, instruction set designing, and carefully crafted data pathways to execute programs efficiently.

This report outlines the design and implementation of a 16-bit CPU, originally inspired by the architecture and design methodology of an existing 8-bit CPU design [1]. The design was developed using VHDL and Quartus schematics and is demonstrated on the Terasic DE10-Lite FPGA board and varies greatly from the reference.

The design extends the 8-bit CPU design by Ben Eater, widening the data path to 16-bits, adopting the Harvard architecture - ROM (Instructions) and RAM (Data) separate, with no shared bus - and has a four-stage pipeline.

The report follows the structure outlined below:

- **Design Specification and Architecture overview**: defines the memory structure, ISA, and function codes for the ALU, ALU2, shift, branch and IO instructions.
- **Component Design and Description**: Explains each component; PC, ROM and RAM, register file, Control Unit, Branch Unit, forwarding unit, ALUs, shifter, pipeline registers, flag latch, and IO including their VHDL or schematic.
- **Top-level Design and Data flow**: Walks through how the components are connected and describes the Fetch, Decode, Execute and Write-back stages, the control signals, hazard handling, clocking.
- **Worked-example Walkthrough**: Details a short program through the pipeline cycle by cycle, covering immediate loads, an ALU instruction with forwarding, a compare, and a branch.
- **Testing and Verification**: describes the VHDL component test benches, the on-board debugging, and the methodology used to verify the capabilities of the CPU
- **Discussion**: evaluate the design decisions, limitations and possible improvements, and summarise the outcomes of the project.

2 Design Specifications and Architecture Overview

2.1 Memory Structures

Before the instruction set Architecture (ISA) is introduced, the memory is defined here.

Data Structure	ADDRESS BITS	SIZE	WIDTH	DESCRIPTION
INSTRUCTIONS	9	512	16	Instruction Memory is seperated from memory and addressable through 9 bits, thus only 512 instruction programs
REGISTERS	3	8	16	Registers hold 8x16 bits of data.
MEMORY	9	512	16	Memory file is seperated from Instruction Memory. We can store up to 512x16 but probably dont need it
SWITCHES	0	1	10	Switches input data, holds 10 bits, only 1 addressable.
SSD	0	1	16	Seven Segment Display output, holds 16 bits, only 1 addressable

Table 1: Data structures used in the CPU

2.2 Instruction Set Architecture

The table 2 is the definitions of the projects instruction set architecture. Even though there are Mnemonics specified for the various $OPCODE$, based on whether the format type further subdivides the Instruction into component functions those functions will have their own Mnemonics to be used in the assembly writing process. Examples of assembly code can be seen in C.1. For example the ALU and ALUI instruction at $OPCODE = 0001$ and $OPCODE = 0010$, are specified with mnemonic ALU and ALUI but contain within them functions with mnemonics ADD, SUB, etc.–Which are the mnemonics used in the Assembly writing process.

The various Instructions (not functions) specified by OPCODE's can be split into different Instruction Types that define the bit mappings they hold, Instruction Types can be re-used for the various instructions to allow ease of implementation and assembly. See more in section 2.2.1. This will show that the OPCODES are all of fixed width, which normally suggests a RISC ISA, which with the limited functionality this design can also be considered RISC or perhaps RRISC – doubly reduced, furthermore the the instruction formats are generally inspired by the MIPS (Microprocessor without Interlocked Pipelined Stages) ISA which aids in the pipeline implementation to reduce any `NOP` insertions or stalls. The Instruction Types are what constrain the Registers to only 8.

The Instruction set was defined as shown in table 2. The design shown has one unused OPCODE. In this design, multiple instructions can be compressed into a single opcode to conserve on space by utilizing the wasted bits of their bit mappings. This will be talked about more later.

OPCODE MAP				
OPCODE	**MNEMONIC**	**FORMAT**	**BITS**	**DESCRIPTION**
0000	NOP	-	16	no operation
0001	ALU	R	16	Loads ALU to select arithmetic/logic R type operation
0010	ALUI	R-I	32	Loads ALU to select arithmetic/logic I type operation
0011	LW	I	16	Load word from memory into register
0100	SW	I	16	Store Word from register into memory
0101	LDI	LI	32	Loads the Immediate number into register. immediate is a 16 bits word
0110				Unused.
0111	BR	BR	16	Branch - jumps somethere only if a flag condition holds
1000	JMP	J	16	Jump - unconditional jump
1001	JAL	J	16	Jump and Link
1010	JR	-	16	Jump Register
1011	SHIFT	R-S	16	Slides bits in the register left or right
1100	ALU2	R	16	Second set of ALU instructions ()
1101	ALU2I	R-I	32	Second set of ALU instructions with immediate
1110	IO	R	16	Interface w/ Switches and SSD
1111	HALT	-	16	Ends program

Table 2: Instruction Set Architecture used in CPU design.

2.2.1 Instruction Types

Table 3 defines the instruction types used and their respective formats and bit allocations that are used in the CPU design.

R Type instructions take 4 operands for operation. The Destination regsiter (rd), the source register (rs or rs1) and another source register (rs2 or rt) along with a function Type of 3 bits–which gives 8 functions. The R-S type instruction is similar to the R-Type instruction except that the third operand is not an address for a register but an amount field that specifies how many bits to shift (0-15)–it is 4 bits, thus leaving only 2 bits for the function–hence it supports 4 functions.

The **I-Type** instruction is used in Memory Access instructions where a rd field and address field are fit into the remaining 12 bits – 3 for register which will either act as a destination or source depending on the type of instruction (SW or LW), and and 9 for the Address field to address the RAM which again can either act as a source or destination depending on the instruction.

The **R-I type** instruction is similar to the R type instruction, except instead of using the 3-bit rs2 field for access to a value in register, a 16-bit field supplied from the instruction memory is used to pass a specified value through. In this case 3-bits are wasted.

The **BR** instruction has a similar format to the I type instruction, however, the CPU's interpretation of the fields is slightly different. There contains a 3-bit *cond* field that specifies the branch function (see table 7). The following 9-bits specify the address that will be accessed on the next clock cycle using the program counter if the condition holds true for a given set of flags.

The **J-Type** instruction type is similar to the branch except there are no conditional flags that are checked meaning 3-bits are wasted. Only the 9-bits are used to specify the address that will be jumped to.

There are **exceptions** with JR, HALT and NOP which contain no bit formats specified. This means they only require 4 opcode bits to execute, which imply 12-bits are technically wasted – In theory these instructions can be compiled into a single ISA slot and differentiated through function bits to make more space for other instructions, however this will be talked about more later. Furthermore, some instructions defined in the ISA within a particular function, such as ***INC rd rs***, or ***NEG rd rs*** or even ***SSD rs***, have specified only 1 destination and source register and only 1 source register respectively, even though the Format is R-Type which imply bit mapping for 2 source registers and a 1 destination register, these are interpretation exceptions within the format, as accounting for all these will make the tables convoluted.

R:	**[15..12] op**	**[11..9] rd**	**[8..6] rs**	**[5..3] rt**	**[2..0] funct**
R-S: (shift)	**[15..12] op**	**[11..9] rd**	**[8..6] rs**	**[5..2] amt**	**[1..0] funct**
I:	**[15..12] op**	**[11..9] rd**	**[8..0] addr**		
R-I	**[15..12] op**	**[11..9] rd**	**[8..6] rs**	**-**	**[2..0]funct**
	[15..0] imm16				
LI:	**[15..12] op**	**[11..9] rd**	**-**		
	[15..0] imm16				
BR:	**[15..12] op**	**[11..9] cond**	**[8..0] imm9**		
J:	**[15..12] op**	**-**	**[8..0] imm9**		

Table 3: Instruction types and their bit formats used.

2.3 From 8-bits to 16-bits

Expanding the instruction width from 8 bits to 16 bits provided greater flexibility in the instruction set architecture (ISA), allowing for more complex operations and a wider range of addressing and register options. Although the original 8-bit design supported 16 possible opcodes, several were unused, and the limited instruction width constrained the amount of information that could be encoded within a single instruction. The 16-bit design retains a 4-bit opcode, preserving the capacity for 16 primary instruction categories, while allocating the remaining 12 bits to operands, register addresses, function codes and immediate values. This enables more versatile instruction formats, including register-based arithmetic and logic operations, bit

shifting, conditional branching, and jump-and-link instructions. Furthermore, 32-bit instructions, formed using two consecutive 16-bit words, support 16-bit immediate values for operations such as loading constants and performing arithmetic with immediate operands. Overall, the expanded instruction width enabled a more capable and flexible CPU design while retaining a straightforward opcode structure and still leaves a lot of room on the table for additional functionality.

2.4 Functions

2.4.1 ALU and ALUI Functions

There are no difference between ALU and ALUI aside from their instruction length, where ALUI is 32-bit and instead of addressing registers for B (rs2) value it instead uses the last 16-bits. The following Functions in Table 4 are shared between them.

ALU and ALUI Functions		
funct	**MNEMONIC**	**DESCRIPTION**
000	ADD	rd <= rs + rt
001	SUB	rd <= rs - rt
010	Increment	rd <= rs + 1
011	Decrement	rd <= rs - 1
100	AND	rd <= rs AND rd
101	OR	rd <= rs OR rt
110	XOR	rd <= rs XOR rt
111	NOT	rd <= NOT rs

Table 4: ALU func codes and respective functions

2.4.2 ALU2 and ALU2I Functions

There are no difference between ALU2 and ALU2I aside from their instruction length, where ALU2I is 32-bit and instead of addressing registers for B (rs2) value it instead uses the last 16-bits. The following Functions in Table 5 are shared between them.

ALU2 and ALU2I Functions		
funct	**MNEMONIC**	**DESCRIPTION**
000	MIN	rd<=min(rs, rt)
001	MAX	rd<=max(rs,rt)
010	MINU	rd<=min(rs, rt)
011	MAXU	rd<=max(rs,rt)
100	NEG	rd <= -rs
101	NAND	rd <= rs NAND rt
110	XNOR	rd <= rs XNOR rt
111	CMP	Null <= rs - rt

Table 5: Table of the functions for ALU2.

2.4.3 SHIFT Functions

In table 6 are listed the various functions and bits associated for the Shift unit.

The four function specified here 7, allows for all types of shifting behavior ROTL is equivalent to ROTR at $O = ROTL(amt) = ROTR(16 - amt)$. And a shift left operation works on both signed and unsigned values– hence no SRA equivalent required.

SHIFT FUNCTIONS		
funct[1..0]	**MNEMONIC**	**Description**
00	SLL	Slide left, fill with zeros
01	SRL	Slide right, fill with zeros Unsigned
10	SRA	Slide right arithmetic - signed
11	ROTL	Rotate left

Table 6: Table of the four shift functions, that take 4-bit amount(0-15)

2.4.4 BR Functions

In Table 7 is listed the various function bits required to branch on certain conditions.

BRANCH CONDITIONS			
cond	**MNEMONIC**	**Branches if**	**Flags**
000	BEQ	Equal	Z = 1
001	BNE	Not Equal	Z = 0
010	BLT	Less Than	N /= V
011	BGE	Greater or Equal	N = V
100	BLE	Less or Equal	Z = 1 || N /= V
101	BGT	Greater Than	Z = 0 || N = V
110	BLTU	Less Than - Unsigned	C = 0
111	BGEU	Greater or Equal - Unsigned	C = 1

Table 7: Funciton Select in branch instrucitons for various conditional jumps.

2.4.5 IO Functions

In table 8 the MNEMONICS capable of use for the IO Instruction are shown.

IO Functions		
funct	**MNEMONIC**	**DESCRIPTION**
000	SSD	SSD <= rs
001	SWI	rd <= SW
010	SSDI	SSD <= [addr] * not Implemented
011	SWI	[addr] <= SW * not Implemented

Table 8: Table of IO Functions. There is a lot more room for development and functionality

3 Component Design and Description

3.1 Program Counter (PC)

The Program Counter used a stateful (memory based)/ combinational unit that updates the output PC based on various control signals from the decode stage. This has the effect that any Program Control/Flow signals that are decoded/executed in the decode stage will immediately effect and update the PC to reduce any bubbles in the pipeline. This allows us to take advantage of this fact and implement the Branch Unit in the decode stage, effectively achieving zero bubble. However a CMP instruction will need to be executed before a branch instruction, so effectively 2 clock pulses are required for a branch but at least this allows us to not have to worry about implementing bubbling.

Figure 1 below shows its functional block implementation. A counter is used with the option of $C_{OUT} = C_{IN} + 1$, or $C_{OUT} = C_{IN} + 2$, this is to alternate between 16 bit instructions or 32-bit instruction since the

Instruction memory is Word (16-bit) Addressable rather than Byte addressable – Otherwise we would have it as +2 and +4. This is by default fed into the DFF (N-bit D Flip Flop) which will store the value upon a rising clock, allowing it be used to update again for C_{IN} in the next clock cycle. The output of this counter is forwarded to both the input of the stack and output as the PC (if selected). The stack would store the PC+1 so that it doesn't enter into an infinite loop as the instruction at that address stored would be a JAL– So JAL then later JR to JAL and so on repeatedly.

The option to ouput the BUS_IN value is also available, this option is selected when the JAL or JMP are high where we need the ouptut to be the bus value, so this fed into the output combination-ally and the input for DFF to be capable of being incremented after the jump.

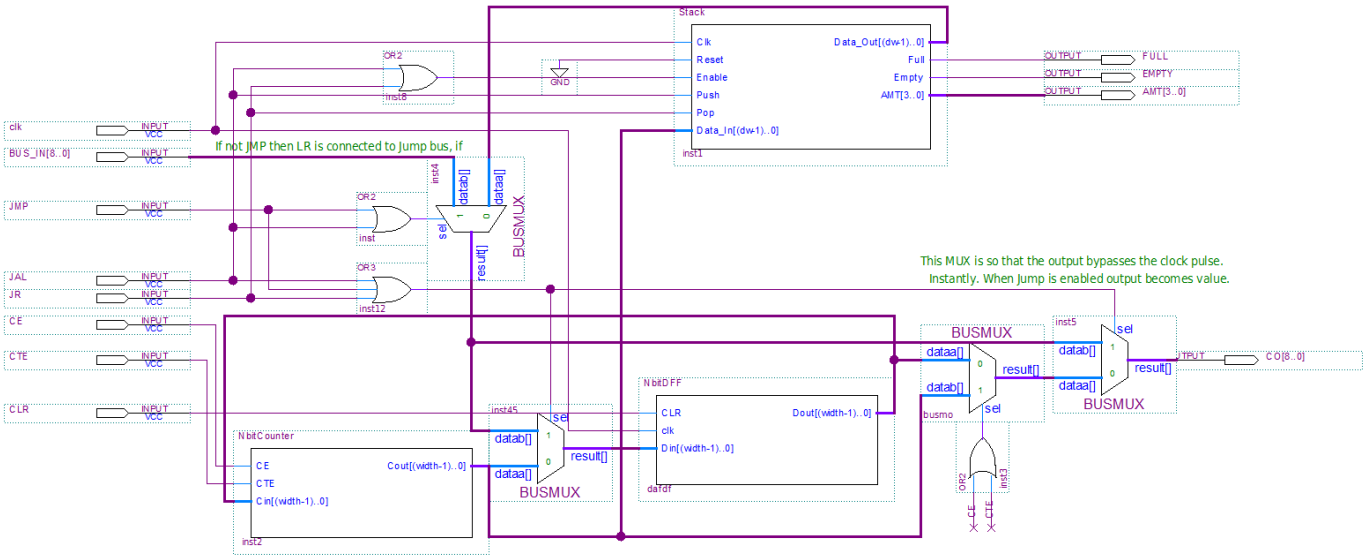


Figure 1: Block Diagram form of the program counter Implementation.

INPUT	DESCRIPTION
clk	Just the clock pulse used in the pipeline stages. This is used to for adding to the stack and setting the N bit DFlip Flop that holds PC of the decode stage.
BUS_IN[15..0]	BUS used for setting the PC to a particular value
JMP	The flag to output the BUS_IN value as the PC. This is for setting the PC to a particular address.
JAL	The flag to ouput the BUS_IN value as the PC but also add the value it was +1/2 to the Stack, so that it can be jumped back to later.
JR	This pops the value at the Stack and uses that value as the PC. Used to go back to a particular point in program.
CE	when this signal is on (without CTE on) the count is enabled by +1
CTE	Count Two Enable. A bit used to increment the PC_{DEC} by 2 instead of 1—This is for 32bit instructions that have an immediate input. When it is set the output is the $PC_{DEC} + 2$
CLR	This is used to clear the D flip flop

Table 9: Description of functionality of the various input signals of Program Counter

OUTPUT	DESCRIPTION
FULL	A bit signal to inform that Stack is full.
EMPTY	A bit signal that informs that Stack is empty.
EMPTY[3..0]	A Vector signal for the amount of elements stack holds.
PC[8..0]	Self-explanatory

Table 10: Descriptions of the various output signals of Program Counter

3.2 Memory (ROM and RAM)

The implementation of Memory in this Basic CPU uses two separate Hardware descriptions for Instruction Memory and Random Access Storage Memory. This decision was made to reduce complexity of implementation. Hence this CPU can be said to be of Harvard Architecture. However modern CPU's already separate their instruction memory and storage memory by the time it gets to the CPU. This concept can be explored in future work however, to implement this we simply need a way to partition the unified RAM which holds both Instruction memory and Data.

3.2.1 Instruction Memory (ROM)

For Instruction Memory define a Read Only Memory (ROM) hardware architecture for its implementation so changes to the program cannot be made on run time. In Figure 2 the ROM entity used is described in VHDL. The ROM is addressable with 9 bits from the input PC. 9-bits gives us up to $2^9 = 512$ instructions capable of being loaded at a time. The ROM outputs 2 16-bit Buses one for PC and one for $PC + 1$ so that the option to use those later 16-bits for an immediate value is always available. In the CPU implementation the ROM is used similarly to a Pipeline Register holding the output for the decode stage effectively also taking the role as Instruction Register. The output of the ROM updates on every rising clock. Of course in a real CPU if we want to run at higher clock frequencies the ROM cannot take both roles and alternative methods should be considered but for a basic implementation this is fine.

As can be seen in the definition we in the architecture we have a comment "–START:" under 'ramdef := (' this is where we can load our program to in hexadecimal or binary.

```
-- This implementation is taken from lecture slides in week 1 - VHDL code examples
entity ROM is
    generic(
        DW: integer := 16; -- The width of the data bus -> 16bit
        AW: integer := 16; -- The address length --> Can be 16 bit but we leave as 7 for 128 instructions
        SIZE: integer := 128
    );
    port(
        ADDR: in std_logic_vector(AW-1 downto 0); -- Address 2
        clk : in std_logic; -- clock signal
        EN: in std_logic; -- Enable

        Dout: out std_logic_vector(DW-1 downto 0); -- Data out
        DoutI: out std_logic_vector(DW-1 downto 0) -- Data out +1
    );
end ROM;

architecture rame of ROM is

    -- Create memory array of size 'SIZE' and width 'WIDTH'
    type ramdef is array (0 to SIZE-1) of std_logic_vector(DW-1 downto 0);

    signal CODE: ramdef := (
        --START:
```

Figure 2: Rom Implementation for Instruction memory

3.2.2 Data Memory (RAM)

This implementation, shown in figure 3 is similar to the ROM implementation except it has both a read and write port with 2 address ports–for Data write and Data access–which only has 16-bit input and output buses. This ROM updates its Read port output every rising edge, and writes the input data on every falling edge. It is used similarly to the ROM instruction memory placed in the in-between stages of the pipeline– In this case it is used in DE/EX intermediary stage, so that data can be accessed as soon as possible, to be forwarded if needed (Otherwise we would need to put a bubble). However, for writes this occurs in the useless Write back (WB) stage, of-course in a real CPU writing and reading is not as instant and can take up many clock cycles without a sophisticated caching hierarchal system. This also is word addressable at 9-bits, hence is capable of storing 512 16-bit words.

```
-- This implementation is taken from lecture slides in week 1 - VHDL code examples
entity RAM is
   generic(
      D_WIDTH: integer := 16; -- The width of the data bus -> 16bit
      A_WIDTH: integer := 16; -- The address length --> Can be 16 bit but we leave as 7 for 128 instructions
      SIZE: integer := 128
   );
   port(
      DIN: in std_logic_vector(D_WIDTH-1 downto 0); -- Data in
      ADDR_IN: in std_logic_vector(A_WIDTH-1 downto 0); -- Address 1
      ADDR_OUT1: in std_logic_vector(A_WIDTH-1 downto 0); -- Read Address 1

      clk : in std_logic; -- clock signal
      EN: in std_logic; -- Enable
      WE: in std_logic; -- write enable

      DOUT: out std_logic_vector(D_WIDTH-1 downto 0) -- Data out bus 1 [ADDR_IN1]
   );
end RAM;
```

Figure 3: Random Access Memory for Program Data storage, similar to a heap

3.3 Registers

Registers are implemented with 2 read ports for simultaneous access of A and B for the Execute stage, and one write port to write the results in write back stage. They are 3-bit addressable at the word level (16-bit) for both input and output. Instructions are written on every falling edge. The registers are not read through edge triggered logic, its accessed combination-ally so does not need to wait–Regardless, if edge-triggered access is preferred it can be replaced for the redundant pipeline register that hold the read data. Writes can only occur if ENI input is set high, this is because it is connected to shared bus delineation is important.

```
entity REGISTERS is -- Two read busses and seperate write bus
   generic(
      AW: integer := 4; -- The width of the address
      W: integer :=16
   );
   port(
      A_ADDR: in std_logic_vector(AW-1 downto 0); -- Address 1 - For reading A
      B_ADDR: in std_logic_vector(AW-1 downto 0); -- Address 2 - For reading B

      W_ADDR: in std_logic_vector(AW-1 downto 0); -- Address 3 - For writing
      DATA: in std_logic_vector(W-1 downto 0); -- Data to write

      ENI: in std_logic; -- Enable Write In
      CLR: in std_logic; -- Clears all registers
      clk: in std_logic; -- Clock signal

      AOUT: out std_logic_vector(W-1 downto 0); -- Output word at A address
      BOUT: out std_logic_vector(W-1 downto 0) -- Output word at B address
   );
end REGISTERS;
```

Figure 4: VHDL entity definition of dual-port Registers

3.4 Control Unit

The control unit, is the functional component that orchestrates everything for the instruction when it is loaded into the decode stage. From the control units perspective the entire instruction executes at that exact instant combinationally at all stages–It is up to the pipeline Registers to segment the control bits for each stage of the pipeline–therefore for a pipe-lined CPU the implementation is surprisingly simple and straightforward, refer to the VHDL code below.

```
-- Control unit is in decode stage and executes combinationally for the instruction within the stage.
entity ControlUnit is
        port(
                OPCODE: in std_logic_vector(3 downto 0); -- The operation that is fetched
                funct: in std_logic_vector(2 downto 0); -- The function of instruction

        -- IF/DECODE STAGE
                Halt: out std_logic; -- if Halt set flush=true and CE=false
                Flush: out std_logic;

                -- PROGRAM COUNTER
                CE: out std_logic; -- Count Enable increments the count on next rising edge
                CTE: out std_logic;
                JMP: out std_logic;  -- Sets the count to what is on its data bus. J Potential optimizat
                JAL: out std_logic; -- Jump and link
                JR: out std_logic; -- Jump to jump register
                BR: out std_logic; -- Branch Instruction
                CLRC: out std_logic; -- count to Clear (straight away) -> no rising edge needed

                MRE: out std_logic; -- Memory read enable (enable memory read in decode stage- to be acc

                --Control pipeline from decode stage
                RDIN: out std_logic; -- enable rd as register read into A

         --FW: out std_logic; -- enable forwarding for the instruction *Depricated: Can just use REN bit
        --1. EXECUTE STAGE
                SWO: out std_logic; -- is IO operation
                EXO: out std_logic; -- select A passthrough in Execute
                IMMO: out std_logic; -- Selects Immediate value into bus
                SHFTO: out std_logic; -- Slect shift unit output to out bus
                ALU2: out std_logic; -- Select ALU2 output to execute bus
                MEMO: out std_logic; -- load instruction, memory out
                IMMBO: out std_logic; --select immediate into B of ALU
        --2. WRITEBACK STAGE
        --2.1. DEVICE SELECTION
                -- REGISTERS
                REN: out std_logic; --Allow write to register
                CLRR: out std_logic;-- Clears the all contents
                -- RAM.
                MEN: out std_logic; -- Memory Write enable
                -- SSD
                SSEN: out std_logic -- Seven Segment Write Enable
        );
end ControlUnit;
architecture orchestration of ControlUnit is
begin
        process(OPCODE, funct)
        begin
```

```
-- Safe defaults for every output, every time.
-- Prevents inferred latches and means each branch
CLRC  <= '0';
CLRR  <= '0';

Flush <= '0';
Halt         <= '0';
CE    <= '1';
CTE          <= '0';
JMP   <= '0';
JAL   <= '0';
JR    <= '0';
BR                <= '0';
MRE        <= '0';
RDIN  <= '0';

SWO        <= '0';
ALU2         <= '0';
EXO   <= '0';
IMMBO <= '0';
SHFTO <= '0';
IMMO  <= '0';
MEMO  <= '0';

MEN   <= '0';
REN   <= '0';
SSEN  <= '0'; --* Depricated: moved to EX

-- below only needs to override what's different.
case OPCODE is
        when "0000" =>  -- NOP
                null;       -- defaults already cover it. Will do nothing in pipeline

        when "0001" =>  -- ALU: rd <= rs, rt
                REN   <= '1';         -- Register File write Enable

        when "0010" =>  -- ALUI: rd = rs, imm16
                CTE <= '1';
                REN   <= '1'; -- Register File write Enable
                IMMBO <= '1'; -- enable immediate operand passthrough

        when "0011" =>  -- LW: rd <= [addr]
                REN   <= '1';        -- Register File write Enable
                MRE         <= '1';
                MEMO         <= '1'; --select load instruction passthrough to bus
        when "0100" =>  -- SW: [addr] <= rd
                MEN   <= '1'; -- Memory write enable
                EXO         <= '1'; -- enable A passthrough
                RDIN         <= '1';

        when "0101" => -- LDI: rd <= imm16
                CTE <= '1';
                REN        <= '1';
                IMMO         <= '1'; --select imm16 passthrough on bus
```

```
                        when "0110" => --CMP
                                null; -- * Depricated

                        when "0111" => -- BR
                                BR          <= '1';
                        when "1000" => -- JMP: IR <= [11..0]
                                JMP    <= '1';
                        when "1001" =>  -- JAL: push[IR], IR <= [11..0]
                                JAL    <= '1';
                        when "1010" =>  -- JR: IR <= pop[IR]
                                JR            <= '1';
                        when "1011" =>  -- SHIFT: rd<= rs<<amt
                                REN           <= '1';
                                SHFTO <= '1';

                        when "1100" => -- ALU2:
                                if funct /= "111" then -- If not CMP Instruction
                                        REN           <= '1';
                                end if;
                                ALU2  <= '1';

                        when "1101" => -- ALU2I:
                                CTE <= '1';
                                if funct /= "111" then -- If not CMP Instruction
                                        REN           <= '1';
                                end if;
                                ALU2  <= '1';
                                IMMBO <= '1'; -- enable immediate operand passthrough

                        when "1110" => -- IO: Instruction register
                                if(funct="000") then
                                        EXO <= '1'; -- Register passthrough to Execute output
                                        RDIN <='1';
                                        SSEN <= '1'; --enable Seven Segment Display
                                elsif funct="001" then
                                        SWO <= '1'; -- Enable switch to bus
                                        REN <= '1'; -- enable register write
                                end if;

                        when "1111" =>
                                Halt <= '1'; -- feeds op '1111' into instruction register (endless ines
                                CE   <= '0';
                        when others =>
                                null;-- undefined opcode: hold safe defaults
                end case;
        end process;
end orchestration;
```

3.5 Branch Unit

The branch unit is located in the decode stage enabling it to combination-ally fire control bits to the Program Counter. This is important as it ensures the Pipeline never needs to stall or bubble. In the execute stage, the branch unit takes inputs from the flag latches and uses them to calculate whether a specific branch condition is met, encoded by another on its inputs. If that condition holds true for those particular set of

flags the Branch Unit will enable BR high and jump to the location specified in the instruction by setting the appropriate flags for the Program Counter. This hence depends on previous instruction to set the flags for its decision. Thus to branch 2 instructions must be used.

Since the branching VHDL is relatively straightforward, the functional block of its use in the decode stage is shown in Figure 5. The CountJump is directly connected to the Program Counter's JMP input.

```
architecture compareTheBranches of BranchUnit is
        signal Z : std_logic; --flags[0] - carry
        signal N : std_logic; --flags[1] - negative
        signal O : std_logic; --flags[2] - zero
        signal C : std_logic; --flags[2] - zero
begin
        O <= flags(0);-- signed overflow
        N <= flags(1);-- Negative flag
        Z <= flags(2);-- Zero condition
        C <= flags(3); -- Carry
   process(Z, N, O,C, cond)-- BRANCH
   begin
                case cond is
                        when "000" => BR <= Z;                  -- BEQ
                        when "001" =>  BR <= not Z;                -- BNE
                        when "010" => BR <= N xor O;         -- BLT (signed)
                        when "011" => BR <= not (N xor O);         -- BGE (signed)
                        when "100" => BR <= Z or (N xor O);         -- BLE (signed)
                        when "101" =>                              -- BGT (signed)
                                BR <= (not Z) and not (N xor O);
                        when "110" => BR <= not C;                 -- BLTU (unsigned)
                        when "111" => BR <= C;           -- BGEU (unsigned)
                        when others =>
                                BR <= '0';
                end case;
        end process;
end compareTheBranches;
```

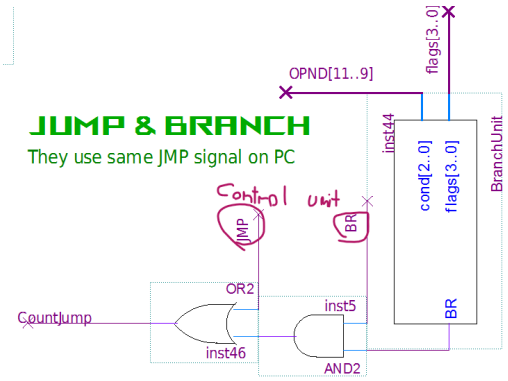


Figure 5: Jump and branch functional block implementation in CPU.

3.6 Forward Select Unit

The forward select unit is used to handle data hazards from data that is needed but has not yet been written to the register files due to the nature of the pipeline. To solve this it is required to feedback the resulting

values from the proceeding stages if and only if they are required by the instruction in the decode stage–The execute stage value has priority over the Writeback (to be fed forward). Code for the logic is shown below:

The functional block diagram of the Forward select Unit is shown in figure 6. The 'FwdSelectUnit' checks either $A(rs1)$ or $B(rs2)$ addresses against the $C(rd)$ addresses of the write-back and execute stages, if it detects a match it outputs bits that funnel those new values back in– through the multiplexers shown. Otherwise, pass through the values read from the register file as there should be no conflict with stale values.

```
architecture fwdCtrl of FwdSelUnit is
begin
        process(exdr,wbdr,na, nb )
        begin
                sela <= '0'; --default: no dependencies dont do anything
                selb <= '0';

                if(FWWB='1') then --If writeback is forwarded
                        if wbdr = nb then-- then this assumes writeback value is the default path
                                selb <= '1';
                                selexb <= '0';
                        end if;
                        if wbdr = na then-- then this assumes writeback value is the default path
                                sela <= '1';
                                selexa <= '0';
                        end if;
                end if;
                if(FWEX='1') then -- since after in the process it will prioritise this if both are on.
                        if exdr = na then --overwrites the previous if
                                sela <= '1';
                                selexa <= '1';
                        end if;
                        if exdr = nb then --overwrites the previous if
                                selb <= '1';
                                selexb <= '1';
                        end if;
                end if;
        end process;
end fwdCtrl;
```

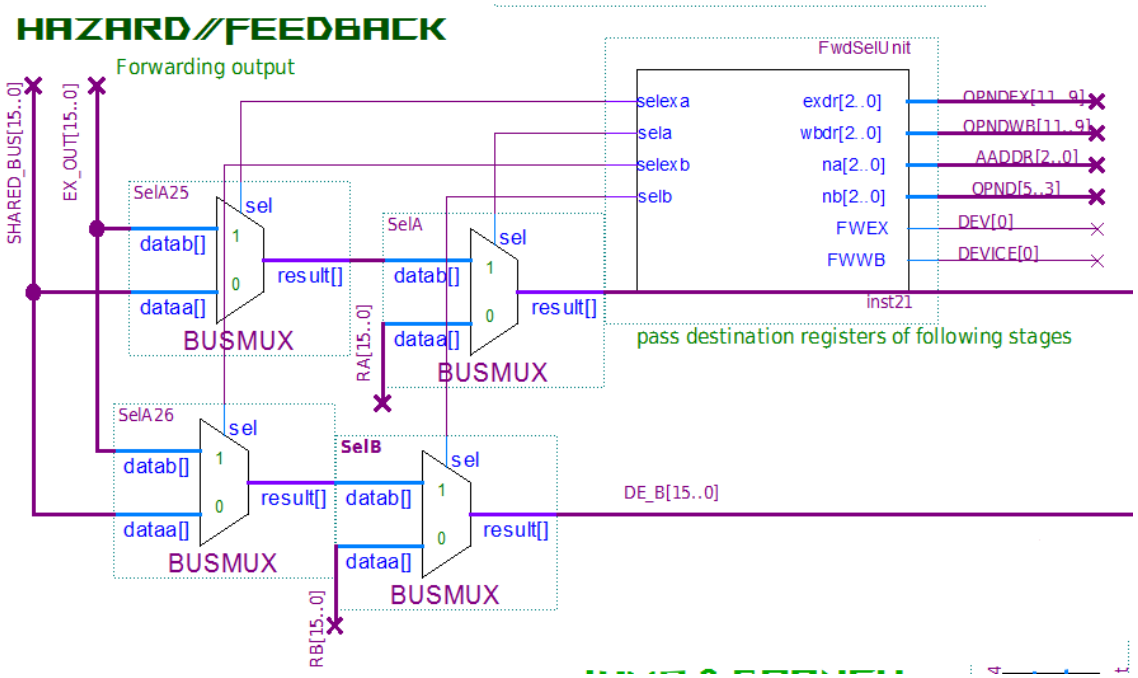


Figure 6: Block Diagram Implementation of Forward select Unit in the CPU.

3.7 Memory Hazards

To handle memory hazards, which occur when an `SW Rx addr1` instruction is followed by an `LW Rx addr1` instruction targeting the same memory address, forwarding is required because the stored value may not yet have been written to memory. This hazard can also occur when one or more instructions execute between the store and load, such as `SW Rx addr1`, followed by instruction (X), and then `LW Rx addr1`.

Since memory is accessed one clock cycle after the decode stage, when the instruction reaches the execute stage, the store instruction may not yet have updated memory with the new value. Consequently, the load instruction cannot reliably obtain the correct data directly from memory and requires forwarding. However, because the forwarded value cannot be made available combinationally across the required clock cycle, it must be stored in a pipeline register for access during the execute stage.

The VHDL implementation below addresses this requirement by storing both the forwarded data and a control signal indicating whether forwarding has occurred. The control signal drives a multiplexer in the execute stage, selecting either the memory output or the forwarded value as the input to the load instruction. This selection is also pipelined to ensure that the forwarded data and its corresponding control signal remain aligned with the instruction.

```
-- Detects data hazards for the SW and LW Writes and reads
-- In the specific scenario where you SW X then follow it with LW X or even seperated by an instruction
entity MemHazUnit is
        generic(
                W: integer := 16;
                AW: integer := 9
        );
        port(
                clk: in std_logic;
                MRE: in std_logic; -- Memory write enable in decode stage
                deaddr: in std_logic_vector(AW-1 downto 0);  -- next read address
                exaddr: in std_logic_vector(AW-1  downto 0);  --  address in Execute stage
```

```
                wbaddr: in std_logic_vector(AW-1 downto 0); -- Current address in Writeback stage (yet t
                -- Determine if the Memory is to be forwarded i.e. instr writes to a register
                MENEX: in std_logic; -- Forward execute stage
                MENWB: in std_logic; -- Forward writebeack stage

                EXO: in std_logic_vector(W-1 downto 0);
                WBO: in std_logic_vector(W-1 downto 0);

                fwded: out std_logic; -- Holds the state, if in the previous clock cycle forwarding occu
                O: out std_logic_vector(W-1 downto 0)
        );
end MemHazUnit;

architecture fwdCtrl of MemHazUnit is
        signal route : std_logic_vector(W-1 downto 0) := (others => '0');
        signal forwarding: std_logic := '0';

        signal fwded_reg: std_logic :='0';
        signal O_reg: std_logic_vector(W-1 downto 0) := (others => '0');
begin
        process(MRE, MENEX, MENWB, deaddr, exaddr, wbaddr, EXO, WBO)
        begin
                route <= (others => '0'); -- By default if no dependency output from the RAM.
                forwarding <= '0';

                if (MRE = '1') then -- Only when we are reading do we really care about dependencies
                        if (MENEX= '1') and (deaddr = exaddr) then --- Latest dependency
                                route <= EXO;
                                forwarding <= '1';
                        elsif (MENWB='1') and (deaddr = wbaddr ) then -- lower priority dependency
                                route <= WBO;
                                forwarding <= '1';
                        end if;
                end if;
        end process;

        process(clk)-- Register the forwarding decision and data together
        begin
                if rising_edge(clk) then
                        fwded_reg <= forwarding;
                        O_reg     <= route;
                end if;
        end process;

        fwded <= fwded_reg;
        O <= O_reg;
end fwdCtrl;
```

3.8 IO (SSD and Switches)

The IO instruction are implemented simply as basic data flow movement in the CPU. For a Switch Input ($SWI\ R1$) instruction we use control bits from the Control unit to funnel the Switch data from switch bus to the execute stages output to later be stored in a specified register. For an Seven Segment Display (SSD R1) function we take the output of the execute stage (which would be just the accessed registers value passed through into execute output) into the writeback stage and simply enable the flag for the register that holds

the output for the SSD. This is shown in Figures 7, and 8.

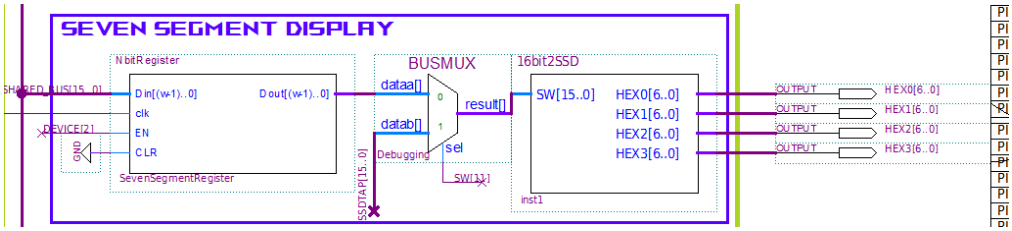


Figure 7: Block Diagram showing a Register connected to the pinout for the seven segment displays. Register is connected to the shared bus. A mux is used to tap into the seven segment display output for on-device-debugging purposes.

3.9 ALU1, ALU2 and Shift Unit (VHDL)

The execute stage holds three combinational units ALU1, ALU2 and the Shift unit, which all receive the same A and B operands and compute their results in parallel on every clock cycle. only one of these results is needed for a given instruction, so the execute output funnel use the `ALU2` and `SHFTO` control bits from the control unit to decide which result is passed through to the execute output. for the immediate variants (ALUI and ALU2I) the `IMMBO` control bit replaces the B operand with the 16-bit immediate held in the second word of the instruction, which is why both units can share the same function codes between their register and immediate forms.

3.10 ALU1

ALU1 implements the eight functions listed in the Table 4 , selected through the 3-bit funct field of the instruction. the most significant funct bit separates the arithmetic operations (ADD, SUB, INC and DEC) from the logic operations (AND, OR, XOR and NOT), and it is this bit that selects which of the two results becomes the output of the unit. The arithmetic is performed on a 17-bit vector so that a carry out is available, however ALU1 does not drive any of the status flags. the flags are produced only by ALU2, and hence the compare instruction was placed in ALU2 rather than ALU1. VHDL Implementation is shown below:

```
entity ALU is
        generic( W: integer := 16 );
        port(
                F: in  std_logic_vector(2 downto 0);
                A: in  std_logic_vector(W-1 downto 0);
                B: in  std_logic_vector(W-1 downto 0);
                O: out std_logic_vector(W-1 downto 0));
end ALU;
architecture alu of ALU is
        signal final: std_logic_vector(W-1 downto 0);
        signal arith: std_logic_vector(W downto 0);         --        17bit output for arithmetic functi
        signal logic: std_logic_vector(W-1 downto 0);-- 16bit output for logic functions
begin
        Process(F, A, B)
        begin
                arith   <= (others => '0');
                logic <= (others => '0');

                case F is -- Both Arithmetic and Logic defined here
                        when "000" => arith <= ('0' & A) + ('0' & B);                          -- ADD: A +
                        when "001" => arith <= ('0' & A) + ('0' & not B) + 1; -- SUB: A - B
```

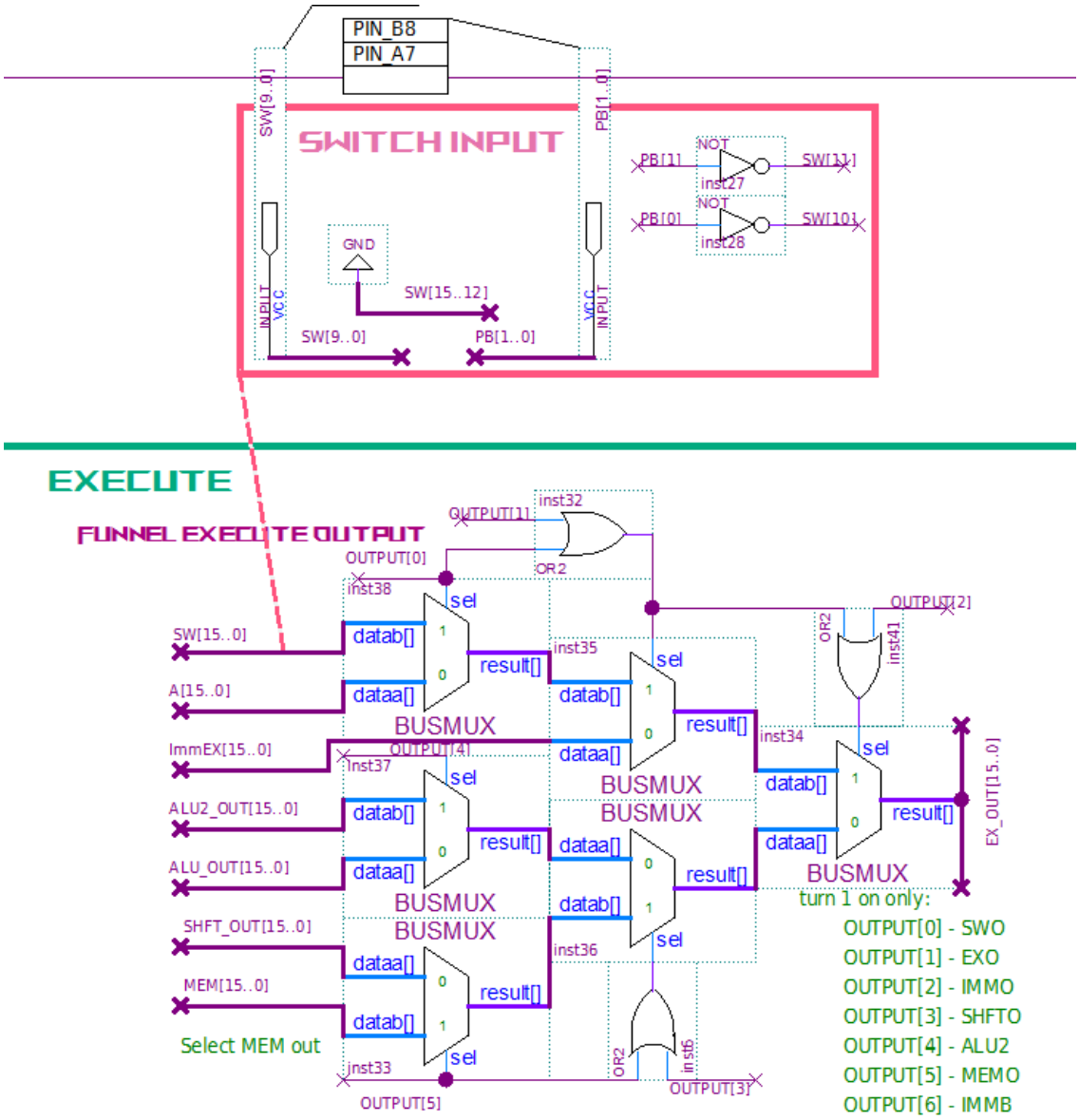


Figure 8: Block Diagram of funnel network for execute output stage and interaction with the FPGA Switch inputs. Ofcourse device technically only has 12 switches the 4 gnd signals are used to make it 16-bit.

```
                        when "010" => arith <= ('0' & A) + 1;
                        when "011" => arith <= ('0' & A) - 1;
                        when "100" => logic <= A and B;
                        when "101" => logic <= A or B;
                        when "110" => logic <= A xor B;
                        when "111" => logic <= not A;
                        when others => arith <=(others => '0'); logic <=(others => '0');
                end case;
        end process;
        -- Select arithmetic or logic result
   final <= arith(W-1 downto 0) when F(2) = '0' else logic;
        O<= final;
        -- No flag output, flags are outputed only on CMP instruction in ALU2
end alu;
```

3.11 ALU2

ALU2 provides the second set of functions shown in Table 5. MIN and MAX compare the two operands as signed two's complement values, whereas MINU and MAXU treat them as unsigned values. NEG produces $-A$ by computing $\overline{A}+1$, and NAND and XNOR are simply the inverted forms of the logic functons already present in ALU1. the final function, `111`, is CMP, which computes $A-B$ as $A+\overline{B}+1$ but does not write a result, since the control unit keeps the register write enable low for this function. the only lasting effect of a CMP is therefore on the flags.

ALU2 generates the four status flags that the branch unit depends on, the negative flag N is the most significant bit of the result, and the zero flag Z is set when every bit of the result is 0. the carry flag C takes the carry out of bit 15 for NEG and CMP and is held at 0 for the other functions. since the substraction is performed as $A+\overline{B}+1$, a carry of 1 indicates that no borrow occured, meaning $A \geq B$ when the operands are treated as unsigned values; this is the condition that BLTU and BGEU test for. the overflow flag V is computed as $(A_{15} \oplus B_{15}) \wedge (A_{15} \oplus R_{15})$, which detects signed overflow in the substraction, and together with N it allows the signed branch conditions BLT, BGE, BLE and BGT to be evaluated.

```
entity ALU2 is -- The second ALU used in the Basic CPU Design
        generic( W: integer := 16 );
        port(
                F:        in  std_logic_vector(2 downto 0);
                A: in  std_logic_vector(W-1 downto 0);
                B: in  std_logic_vector(W-1 downto 0);

                O: out std_logic_vector(W-1 downto 0); -- Output
                V: out std_logic;        -- Overflow
                N: out std_logic; -- Negative flag
                Z: out std_logic; -- Zero flag
                C: out std_logic         -- Carry flag
        );
end ALU2;
architecture alyou2 of ALU2 is
        signal final: std_logic_vector(W-1 downto 0);
        signal arith: std_logic_vector(W downto 0);
        signal logic: std_logic_vector(W-1 downto 0);
begin
        Process(F, A, B)
        begin
                arith   <= (others => '0');
                logic <= (others => '0');
```

```
                case F is -- Both Arithmetic and Logic defined here
                        when "000" =>
                                if signed(A) < signed(B) then logic <= A;
                                else logic <= B;
                                end if;
                        when "001" =>
                                if signed(A) > signed(B) then logic <= A;
                                else logic <= B;
                                end if;
                        when "010" =>
                                if unsigned(A) < unsigned(B) then logic <= A;
                                else logic <= B;
                                end if;
                        when "011" =>
                                if unsigned(A) > unsigned(B) then logic <= A;
                                else logic <= B;
                                end if;
                        when "100" => arith <= ('0' & not A) + 1;                -- NEG: -A

                        when "101" => logic <= not (A and B);                        -- NAND:
                        when "110" => logic <= not (A xor B);                        -- XNOR:

                        when "111" => arith <= ('0' & A) + ('0' & not B) + 1;         -- SUB: A - B (dis

                        when others => arith <=(others => '0'); logic <=(others => '0');
                end case;
        end process;
        -- Select arithmetic or logic result
        final <= arith(W-1 downto 0) when (F = "100" or F = "111") else logic; -- Select between logic o
        O <= final; -- output the selection
        -- FLAGS
        V <= (A(W-1) xor B(W-1)) and (A(W-1) xor final(W-1)); -- conditions for signed overflow
        C <= arith(W) when (F = "100" or F = "111") else '0'; -- Carry: Only when arithmetic operation o
        N <= final(W-1);
        Z <= '1' when final = (final'range => '0') else '0';  -- Zero
end alyou2;
```

3.12 SHIFT Unit

The shift unit is used by the R-S instruction format. it takes the A operand together with a 4-bit amount field from bits [5..2] of the instruction, giving shifts of 0 to 15 places, and a 2-bit function from bits [1..0] as listed in Table 6. the shifts themselves are implemented with the `shift_left`, `shift_right` and `rotate_left` functions of the `numeric_std` library. SRL fills the vacated upper bits with zeros, while SRA fills them with copies of the sign bit so that a negative value remains negative after the shift. Only left rotate is provided, as right rotate by n places gives the same result as a left rotate by $16 - n$ places.

```
entity ShiftUnit is
        generic(
                W : integer :=16);
        port(
                F: in std_logic_vector(1 downto 0); -- Funct select type of shift operation
                A: in std_logic_vector(W-1 downto 0); -- Input A
                amt: in std_logic_vector(3 downto 0); -- amount of bits to be shifted

                O: out std_logic_vector(W-1 downto 0) -- Ouptput the shifted
```

```
        );
end ShiftUnit;
architecture shifty of ShiftUnit is
begin
        process(F, amt, A)
        begin
                case F is
                        when "00" => -- SLL
                                O <= std_logic_vector(shift_left(unsigned(A), to_integer(unsigned(amt))));
                        when "01" => -- SRL
                                O <= std_logic_vector(shift_right(unsigned(A), to_integer(unsigned(amt))));
                        when "10" => -- SRA
                                O <= std_logic_vector(shift_right(signed(A), to_integer(unsigned(amt))));
                                O(W-1) <= A(W-1);
                        when "11" => -- ROL
                                O <= std_logic_vector(rotate_left(unsigned(A), to_integer(unsigned(amt))));
                        when others => O <= (others => '0');
                end case;
        end process;
end shifty;
```

3.13 Register, Pipeline Register and Flag Latch

3.13.1 Register File

The register file holds eight 16-bit general-purpose registers, R0 to R7, which are addressed through 3 bits. Each register is an `NBitRegister` composed of sixteen sinle-bit flip-flops (`BitRegister2`), each with a write enable and an asynchronous clear. For a write, an `NBitDEMUX` decodes the write address into a one hot set of enable lines, gated by the `ENI` write enable, so that only the addressed register is loaded with the data on its input. For reads, two separate 8:1 multiplexers (`NBit_1MUX`) select the registers addressed by `A_ADDR` and `B_ADDR`. As these reads are combinational, both operands are available within the same cycle without having to wait on a clock edge. The registers are written on the falling edge of the clock, whereas the pipeline registers capture their inputs on the rising edge. A result written back during the write-back stage is hence already stable in the register file before the next rising edge occurs.

3.13.2 Pipeline Registers

The pipeline stages are separated by `NBitPR` pipeline registers, which are generic-width registers that capture their input on the rising edge of the clock. Each has a stall input, `STL`, that holds the current value while it is high, along with a generic `CLKN` that allows the register to update only once every N clock cycles. Every instance in this design uses a `CLKN` of 1 and the stall input is left unused, since the forwarding nit and the decode stage branch unit together remove the need to stall the pipeline. The instances used in the top-level design are listed in Table 11.

Instance	Width	Contents
ImmeRegister	16	Immediate word (imm16) passed from decode to execute
ExecuteOperands	12	Register addresses and funct bits for the execute stage
ABuffer	16	A operand after forwarding
BBuffer	16	B operand after forwarding
OUTPUTSELECT	7	Select bits for the execute output funnel
Executed Output	16	Execute result passed to write-back
WritebackOperands	12	Destination address and control bits for write-back
DEVICESELECT	3	write-back device enables (resgister, RAM, SSD)
DeviceSelregister	3	Second stage of the device enables

Table 11: Pipeline registers used in the top-level design.

3.13.3 Flag Latch

The four flags produced by ALU2 are held in a 4-bit level-sensitive latch (`NBitLatch`) rather than an edge-triggered register. While its enable is high, the latch is transparent, so the flags generated by a CMP in the execute stage reach the branch unit in the decode stage within the same clock cycle. Once the enable falls, the latch holds those flags for any subsequent branch. This is what allows a branch to directly follow its CMP without stalling the pipeline. Latches are generally avoided in FPGA designs as they complicate timing analysis; however in this case Quartus reports it as free of timing hazards, and for a basic implementation this approach was considered acceptable.

3.14 Supporting Components

3.14.1 Demultiplexer

`NbitDEMUX` is a generic decoder with an n-bit address input and 2^n outputs. when it is enabled input is high, the output selected by the address is driven high and all remaining outputs are held low, and when the enable is low every output is held low. within the register file it is used with a 3-bit address to convert the single write enable into the eight individual register enables.

3.14.2 Multiplexers, Buses

`NbitN_1MUX` is a generic 2^n:1 multiplexer whose inputs are packed into a single wide vector, with the select value choosing which 16-bit slice of that vector is passed to the output. Two of these form the read ports of the register file. The remaining 2:1 selections on the 16-bit buses in the top-level schematic are made with Quartus `BUSMUX` blocks, which include the forwarding multiplexers in front of the A and B operands and the immediate select on the B input of the ALUs. An `NBit_1MUX` is also used to select which debugging value is shown on the seven-segment displays, as described in Table 18.

4 Top-Level Design and Data Flow

4.1 Top-level Pipelined Architecture

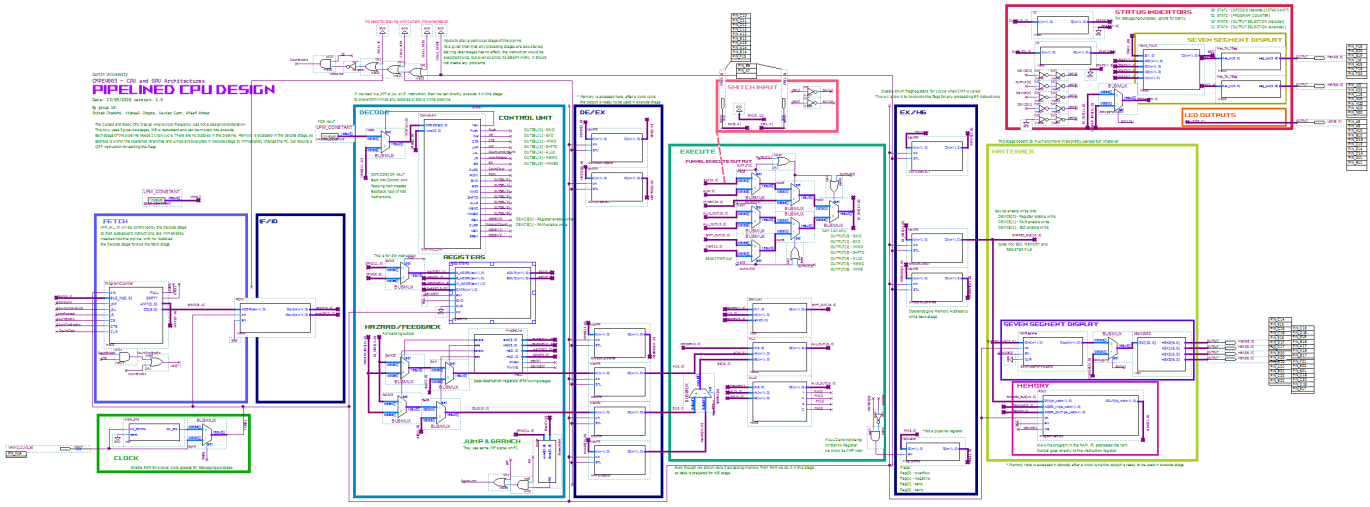


Figure 9: Top Level view of CPU design. View of Stages and Instruction Flow.

The processor designed in this project is a 16-bit CPU implemented in Quartus with the Intel Max-10 FPGA board. The control unit and ALU are designed utilising VHDL, whilst the remaining blocks are a combination of VHDL and schematic-logic. The design is a reinvention of the 8-bit reference CPU provided with every register, bus, memory word, and ALU operand being 16-bits wide, with ALUI1 and ALUI2 being 32-bits wide. The Instructions are either one 16-bit word or 32-bit words (2 16-bits).

The pipeline implemented within the design breaks up the instructions into Fetch, Decode and Execute separated by each of the pipelines IF/ID, DE/EX, EX/WB. A writeback block is included in the schematic, it is the final stage of instruction execution it saves the final result of a given instruction into a destination register. The pipelines maximises the processing power of the CPU by allowing multiple instructions to run simultaneously across different stages without waiting for a full cycle to finish - while one executes, another decodes, and another fetches. An example of the concurrency of the instruction pipeline can be shown in Table 12.

4.1.1 Fetch Stage

The Fetch stage contains the Program Counter and the instruction memory etch. The PC is built in schematic from an N-bit counter, an N-bit D Flip-Flop Register, a hardware return-address stack, and bus multiplexers. For control of the PC it has the inputs for Count Enable (CE), Count-two Enable (CTE), Clear (CLR), Jump (JMP), Jump and Link (JAL), and the Jump Register (JR). The 9-bit output of the counter CO[8..0] helps form the address of the instruction memory.

The instruction memory is a synchronous ROM designed in VHDL for parameters of: data width 9 (DW=16), address width (AW), and size (= 128). On a rising clock edge when enable (EN) is high the ROM will distribute two outputs: Dout to Address (ADDR) and DoutI to ADDR + 1. Dout is the instruction and DoutI is the second word of a two-word instruction, both of these are fetched in the same cycle as the opcode. The ROM output register essentially acts as the IF/ID pipeline register.

4.1.2 Decode Stage

Control Unit The control unit reads the opcode provided to it and asserts signals according to the input. It has the inputs OPCODE[3..0], funct[2..0] and also has a halt input to cease all executing any further in-

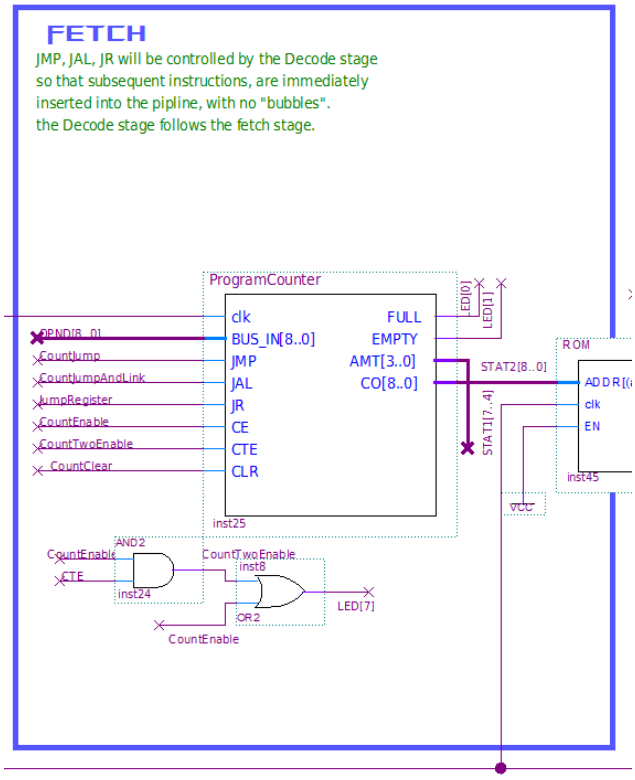


Figure 10: Block Diagram implementation of the fetch stage, showing PC integration.

structions. A multiplexer feeds into the control unit and when a halt instruction is inserted it will consistently feed a halt instruction and the processor stays halted until reset.

Register The register block in the decode stage holds 16 addressable read-write registers each with 4-bit addresses. For writing, an N-bit Demultiplexer (NBitDEMUX) decodes the write address into a set of enable lines where exactly one line is logic 1 whilst the others are logic 0, controlled by write enable (ENI), which ensures only the addressed register loads the input data. For read, two separate 16:1 multiplexers (NBitN_1MUX) select the registers addressed by A_ADDR (Address 1) and B_ADDR (Address 2). These reads can be combined, meaning both operands are processable in the same cycle without waiting for a clock edge - the registers are written on the rising clock edge.

Branch Unit The branch unit recieves the four stored flags and a condition code and produces output (BR). This AND,ed with the Control Unit's BR and OR,ed with the CountJump (from the Control Unit), so conditional branches and unconiditional jumps share the single input of the PC JMP. The flags are being checked by the branch and are saved from whichever Compare (CMP) instruction ran before it.

4.1.3 Execute Stage

The execute stage contains two ALU's mentioned previously; ALU and ALU2. One is a shifter and the other is an output funnel that selects the result.

ALU2 and ALU2I similar to the above only differ in instruction length

Shift Unit The shift unit take an operand (i.e OperandA), a 4-bit shift amount and a 2-bit mode and produces the output (SHFT_OUT). The ROM program uses SSL (00) and SRL (00) to have precise control over bits in the register.

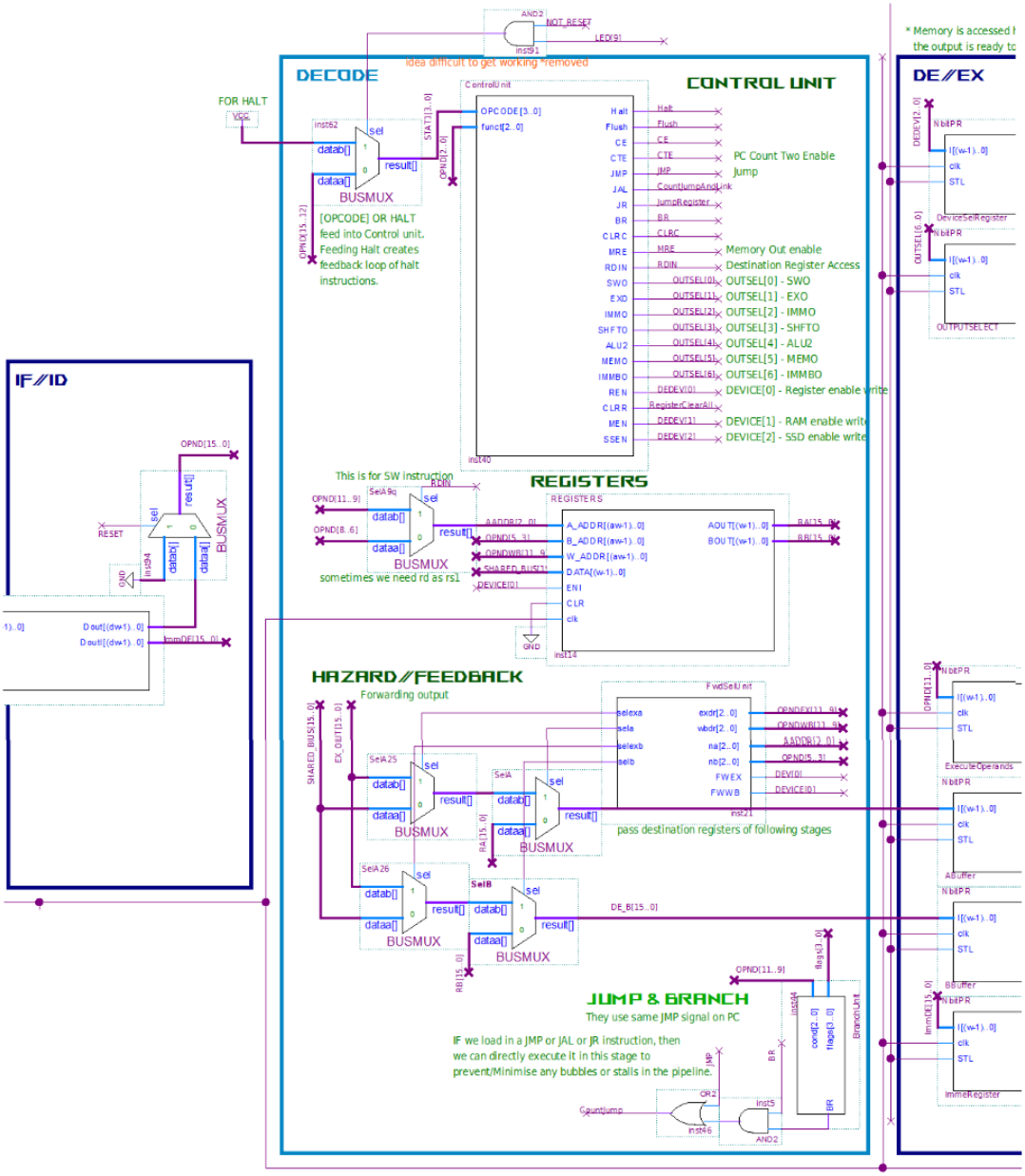


Figure 11: Close up block Diagram of Decode stage of the pipeline with all functional component interactions

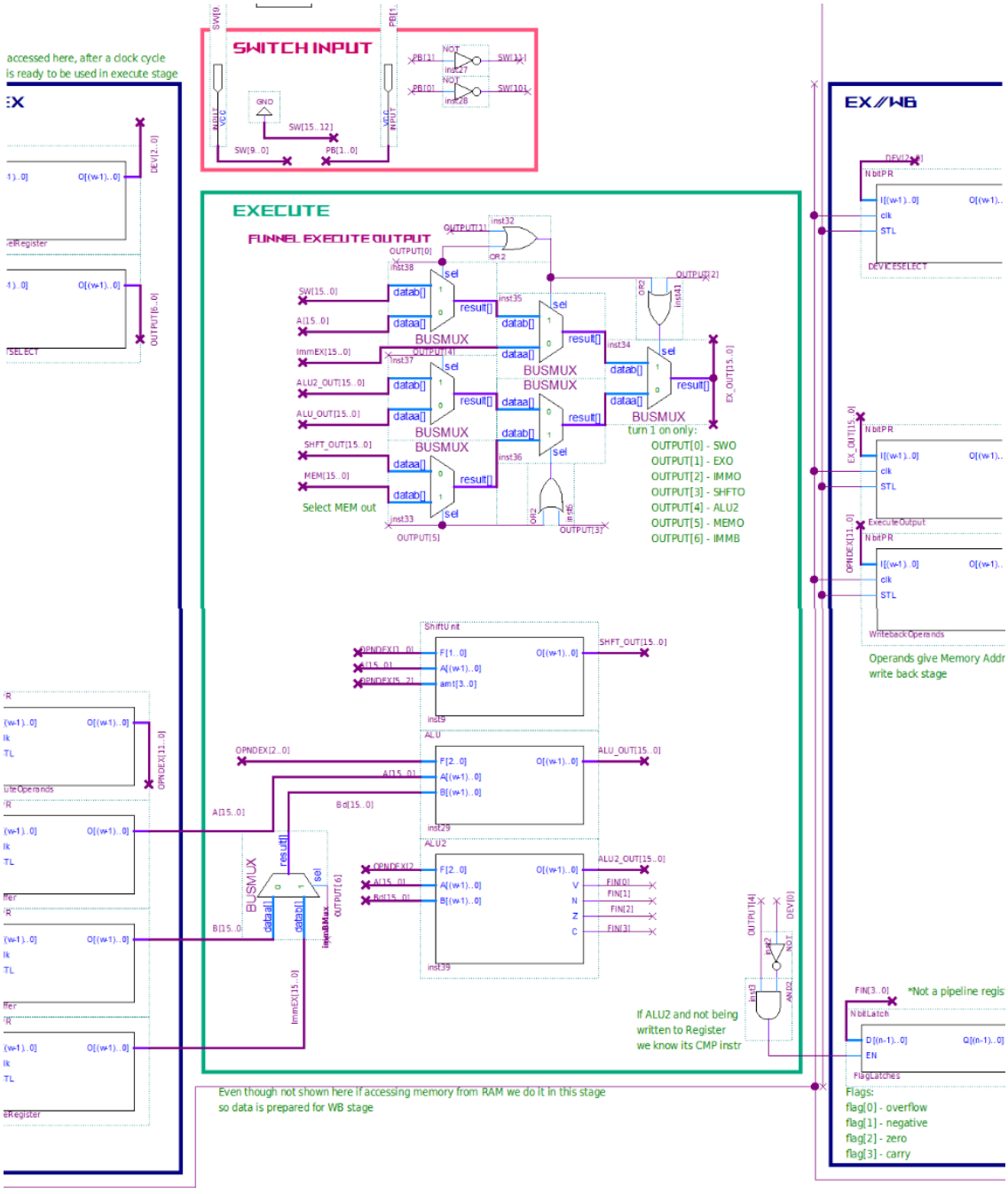


Figure 12: Close up block diagram implementation of Execute Stage of Basic CPU,

Result selection and Status Flags The results feed a of multiplexers driven by the one high line (discussed earlier in the register) so exactly one source reaches out (EXOUT[15..0]). OUTSEL[4] then chooses between ALU2 output or ALU depending on what the preference for data is.

ALU2 produces the four flags: Overflow, Negative, Zero, and Carry. All of which are assigned either 1 or 0 and fed as FIN[3..0] into the FlagLatch which outputs Flags[3..0]. The flaglatch is not a pipeline register and feeds its output to the Branch Unit in Decode.

4.1.4 Writeback Stage

The EX/WB registers hold the Execute result EX_OUT, the write-back operands (destination register or memory address), and the DEVICE[2..0] enable bits,.

Device[0] - Register enable write Device[1] - RAM enable write Device[2] - SSD enable write

The result is distributed as SHAREDBUS[15..0] to the Seven Segment Display (SSD), Memory, and the Register File with the enable bits deciding which captures the value.

- **Register file** (`DEVICE[0]`): `SHARED_BUS` drives `DATA`, the destination field drives the write address, and `ENI` is applied. The write happens on the falling clock edge.
- **Data RAM** (`DEVICE[1]`): `SHARED_BUS`, the write-back operand drives `ADDR_IN`.
- **Seven-segment display** (`DEVICE[2]`): `SHARED_BUS` is captured by the seven-segment register. A `16bit2SSD` decoder converts it to four hexadecimal digits on `HEX0`–`HEX3`. A switch-controlled multiplexer can bypass the register for debugging.

Pipeline Registers The stages (Fetch,Decode,Execute,Writeback) are separated by (NBitPR) pipeline registers. Each are registers that capture each stages input on the rising clock edge. Each of which hold a stall input (holding current value while high) and a (CLKN) that allows the register to update once every N clock cycles.

Hazard handling and forwarding An instruction in the Decode phase may need a register that an earlier instruction hasn't written yet. The FwdSelUnit compares the source register numbers with the destination registers in the DE/EX and EX/WB stages and produces four select lines: selexa, sela, selexb, selb. The select lines trigger the multiplexers SelA25, SelA, SelA26, Selb, which choose between the register-file output, EXOUT (which is one instruction ahead) and SHAREDBUS (two ahead), with the nearest result getting priority in selection. It allows dependent instructions to run back to back without stalls.

Clocking The pipeline registers share one clock, chosen by a multiplexer which rests between the 50MHz clock and divided clock (`clk_2hz`). Pipeline registers capture on the rising edge, and the register file is written on the falling edge.

4.2 Worked example walkthrough

At reset the program counter is PC = 0, pointing to the first instruction in memory, and all other pipeline stages are empty (Figure 14). Table 12 summarizes the pipeline contents on each clock cycle.

4.2.1 I-Type Load Instructions

On the first rising clock edge, `LDI R1 31` is fetched. Because it is a 32-bit instruction (the immediate occupies a second word), the PC is incremented by 2 and now points to address `0x002`. On the next edge, `LDI R1 31` moves into Decode, where `REN` and `IMMO` are set high; these control bits are buffered in the pipeline register and used by later stages. Meanwhile, `LDI R4 38` is fetched (Figure 14).

On the following edge, `LDI R1 31` reaches Execute, where `IMMO` routes the immediate value straight to the Execute output bus. `REN` is carried forward so that the value can be written back later (Figure 15).

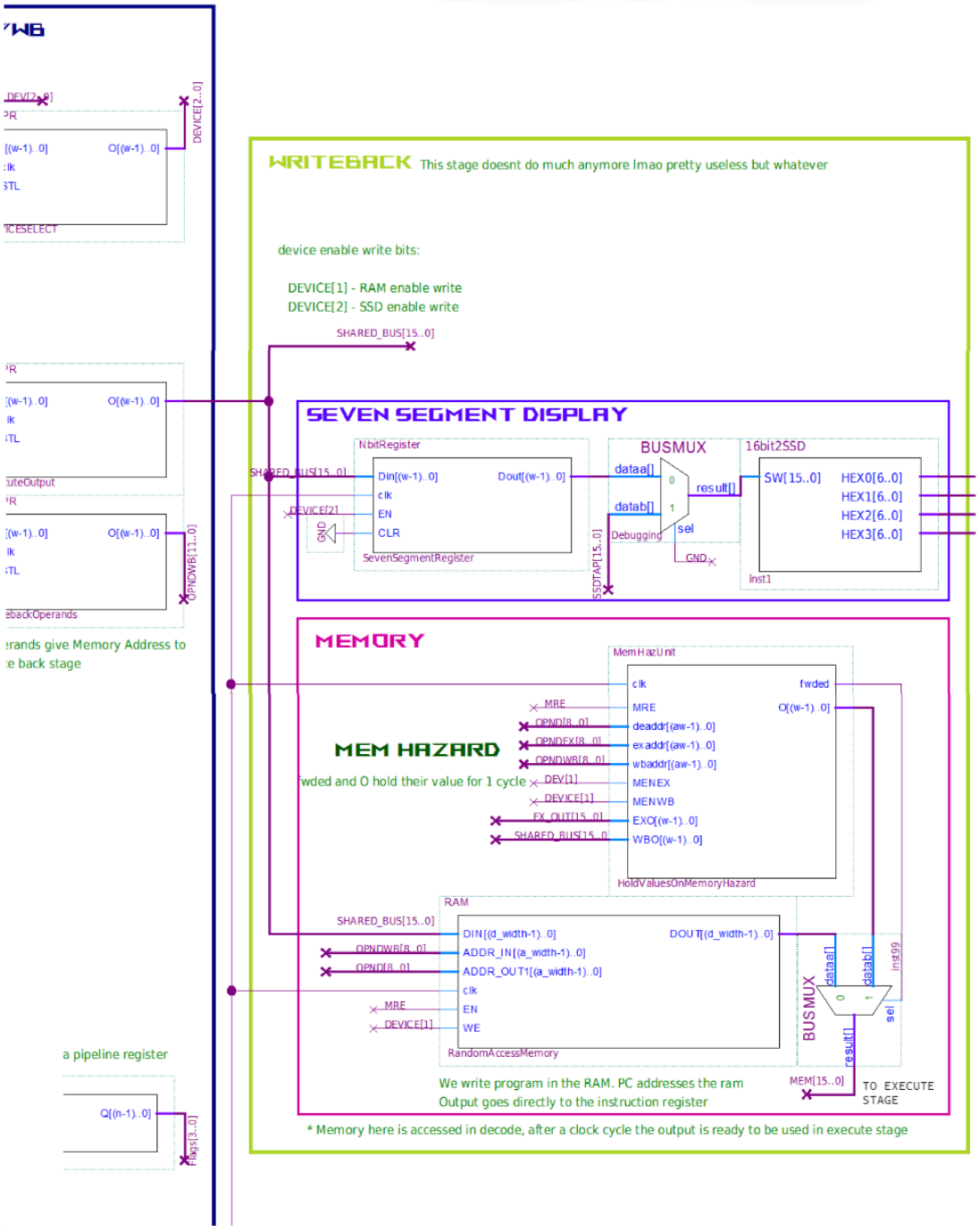


Figure 13: Close up block diagram of Write back Stage.

CPU Worked Example						
Clk	**PC**	**IF**	**DE**	**EX**	**WB**	**Ref**
1	**0**	**LDI R1 31**	**-**	**-**	**-**	Figure 14
2	**2**	**LDI R4 38**	**LDI R1 31**	**-**	**-**	Figure 14
3	**4**	**ADD R2 R4 R1**	**LDI R4 38**	**LDI R1 31**	**-**	Figure 15
4	**5**	**SSD R2**	**ADD R2 R4 R1**	**LDI R4 38**	**LDI R1 31**	Figure 16
5	**6**	**CMP R2 0x45**	**SSD R2**	**ADD R2 R4 R1**	**LDI R4 38**	Figure 17
6	**8**	**BEQ 1**	**CMP R2 0x45**	**SSD R2**	**ADD R2 R4 R1**	Figure 18
7	**1**	**NOP**	**BEQ 1**	**CMP R2 0x45**	**SSD R2**	Figure 19
8	**2**	LDI R4 38	**NOP**	**BEQ 1**	**CMP R2 0x45**	None
9	**4**	ADD R2 R4 R1	LDI R4 38	**NOP**	**BEQ 1**	
10	**5**	SSD R2	ADD R2 R4 R1	LDI R4 38	**NOP**	

Table 12: Worked example Tabular summary through clock cycles

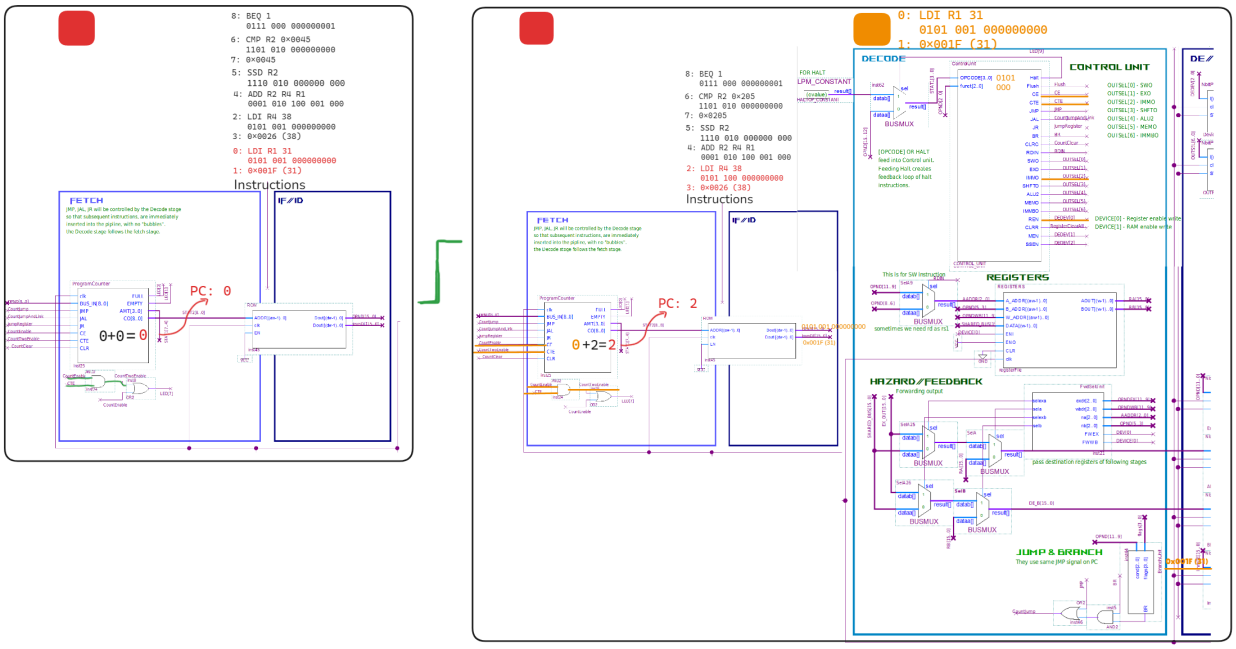


Figure 14: The PC addresses the ROM at 0. On the rising clock edge, the instruction is fetched and passed to the Decode stage. As the instruction is 32 bits long, the PC is set to its current value +2.

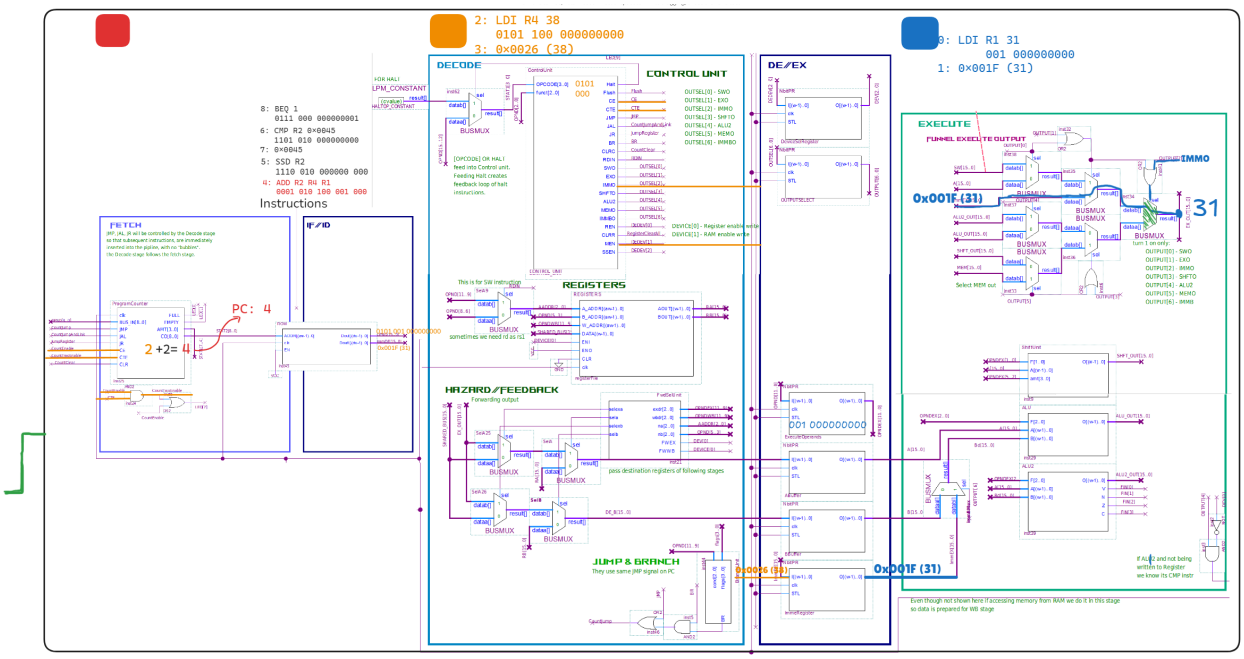


Figure 15: An `LDI` instruction in the Execute stage simply routes its immediate value to the Execute output.

4.2.2 R-Type ALU Instructions

When `ADD R2 R4 R1` reaches Decode, `REN` is set high for use in Writeback. The instruction depends on both `R1` and `R4`, neither of which has yet been written to the register file. The `FwdSelUnit` therefore forwards the pending values from the Execute and Writeback stages into the pipeline registers for operands A and B. Since this instruction is 16 bits wide, the PC increments by only 1 (Figure 16).

On the next rising edge, `ADD` executes and its result is placed on the Execute output, while `LDI R4 38` is in Writeback and, with `REN` high, is about to write its immediate into `R4`. `SSD R2` is now in Decode and needs `R2`, which has not yet been written; the value is forwarded from the Execute stage, where the addition is computed combinationally (Figure 17).

4.2.3 J-Type Program Flow Instructions

`CMP R2 0x45` is now in Decode, setting the `ALU2O` and `IMMB` control signals high so that the constant `0x45` is passed through. As `R2` is still outstanding, the `FwdSelUnit` bypasses the value from Writeback into the A pipeline register. Meanwhile, `SSD R2` is in Execute, where `EXO` is set high so that register A (`R2`) passes through to the Execute output, and `ADD R2 R4 R1` is in Writeback, writing its result into `R2` on the next clock edge (Figure 18).

On the next clock cycle, `SSD R2` reaches Writeback, where `SSEN` is set high and the value on the bus (`R2`) is written to the SSD register. `CMP` executes and its flags are latched immediately. `BEQ`, now in Decode, evaluates those flags in the same cycle and, since the branch is taken, sets the PC to 1 (Figure 19).

Address 1 holds the immediate of the first instruction (31, i.e. `0x001F`), which is now fetched and decoded as an instruction. It is interpreted as a `NOP`, which simply increments the PC to 2. Execution therefore resumes from address 2 and the program repeats in a loop. `BEQ` does nothing in the later stages, and `CMP` does nothing in Writeback.

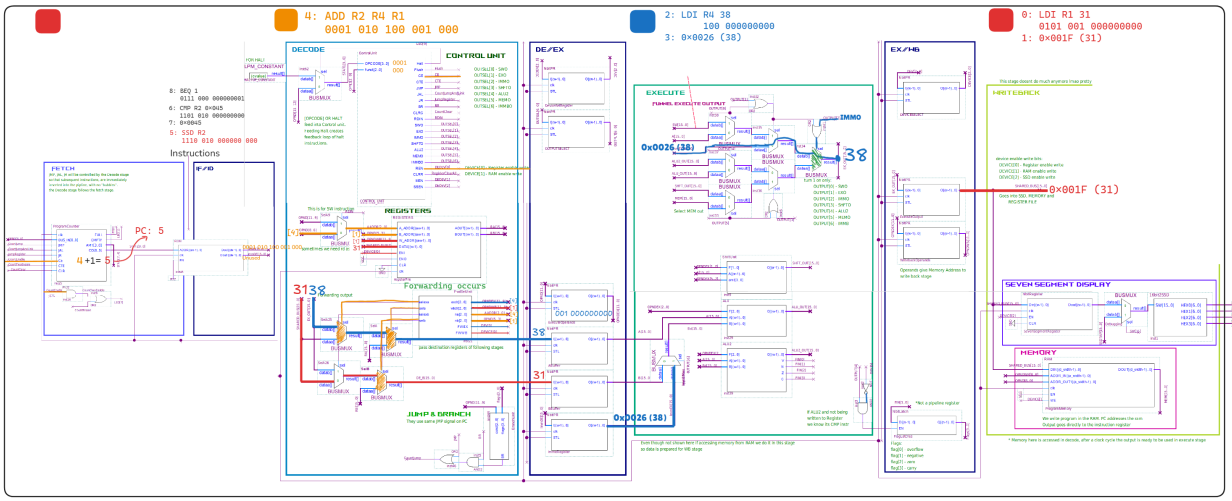


Figure 16: The `ADD` instruction needs the values of registers that have not yet been written back, so they are forwarded into the Decode stage.

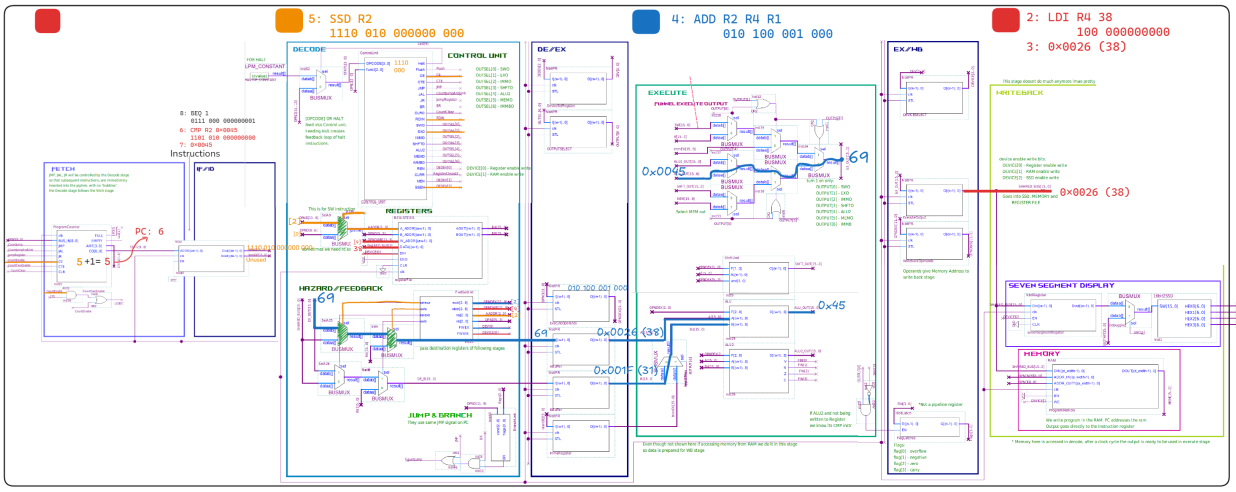


Figure 17: `SSD R2` in Decode receives the result of `ADD` forwarded from the Execute stage.

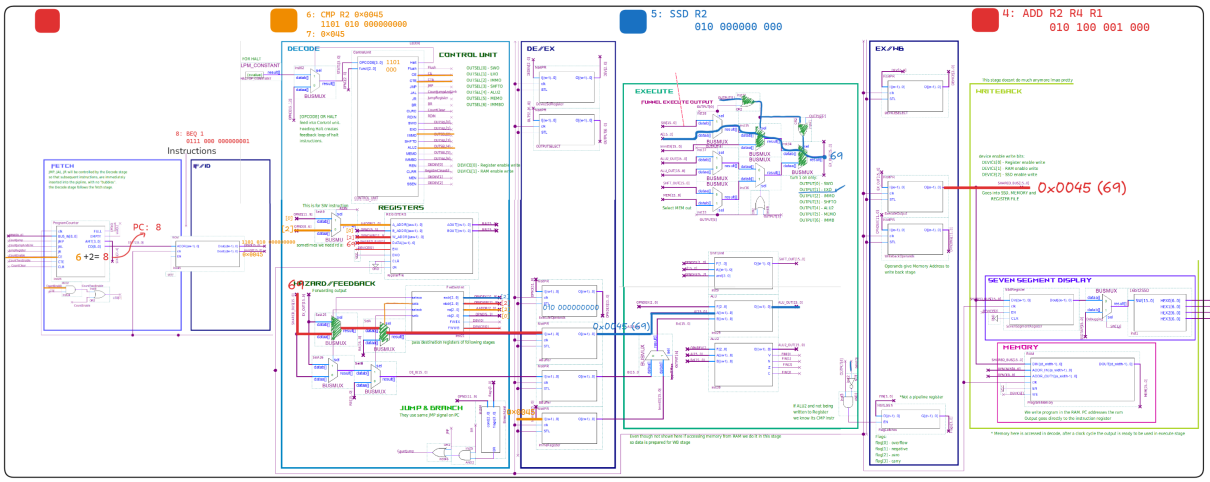


Figure 18: `BEQ 1` is fetched while `CMP` is in Decode.

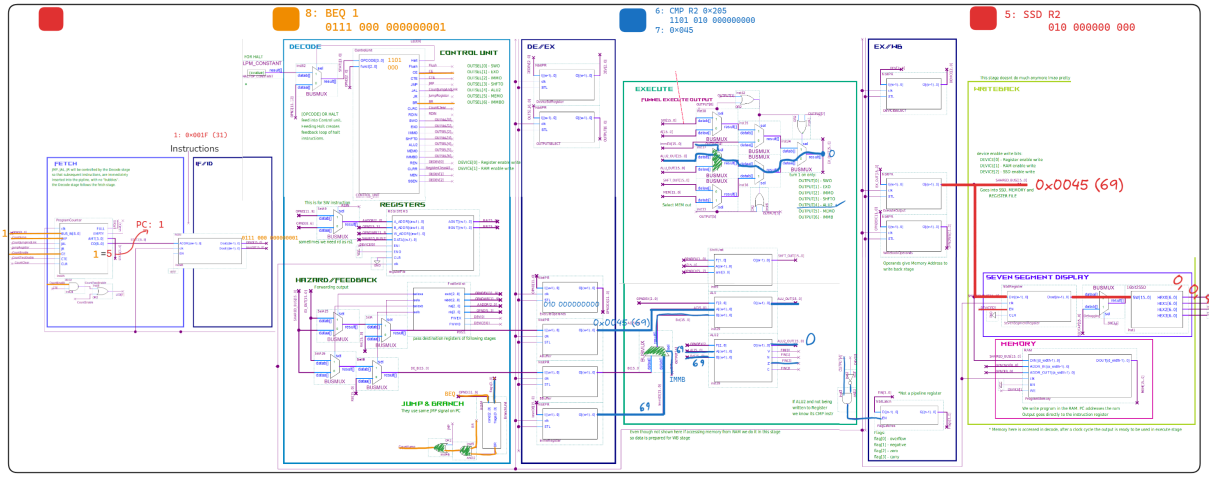


Figure 19: `BEQ` has executed and the branch is taken, setting the PC to 1.

4.3 Clock cycles per instruction

Every instruction occupies the decode stage for a single clock cycle. The 32-bit instructions do not need an additional cycle, as the ROM outputs the words at PC and PC+1 together and the program counter advances by 2 in the same cycle. Jumps and branches are resolved in the decode stage, and since the program counter is combinational, the following fetch is already taken from the target address, so no bubble is inserted into the pipeline. The number of cycles for each instruction type is summarized in Table 13, some instructions only require one stage for completion, but since they all stay-in/occupy the pipeline for 3 cycles they are said to take 3.

Instruction	**Words**	**Cycles**	**Notes**
NOP, ALU, ALU2, SHIFT, IO	1	3	
ALUI, ALU2I, LDI	2	3	PC advances by 2
LW, SW	1	3	RAM read in DE/EX, write in WB
JMP, JAL, JAR	1	3	Resolved in decode, no bubble
BR	1	3	Requires a preceding CMP
HALT	1	-	Stops the program counter

Table 13: Clock Cycles per instruction type.

Once the pipeline is filled with 3 instructions, the CPU therefore completes one instruction every clock cycle, giving a CPI of 1. The exception is a conditional branch, which effectively costs two cycles due to the CMP that must precede it.

5 Testing and Verification

For the simpler components creating test benches with VHDL with inline assert statements for fast test case checking–the Block Diagram implementations needed to be converted to VHDL for this. These test benches were run on QuestaSim to observe the failure logs, and the waveforms were inspected for debugging. But this Method generally wasn't used extensively for testing as opposed the following.

Since creating a VHDL test bench for the Basic CPU seemed overwhelmingly annoying, instead a combination of a bunch of Status Indicators connected to LEDs and Seven Segment Displays along with a slow 0.5Hz clock was used to slowly inspect and debug the uploaded CPU. Tables were created for easy access

5.1 Component Test benches

For the simpler components, test benches were written in VHDL with inline `assert` statements, so that each test case reports a failure automatically rather than requiring the waveforms to be inspected by hand. Where a component had been designed as a block diagram, it first had to be converted to VHDL before a testbench could be written for it. The testbenches were run in QuestaSim, where the transcript was checked for failed assertiions and the waveforms were used to locate the cause of any failure. The testbenches written and the behaviour each one checks are listed in Table 14.

Testbench	Asserts	Behaviour checked
tb_ALU	8	Each of the eight ALU1 functions (ADD, SUB. INC, DEC, AND, OR, XOR, NOT)
tb_ALU2	11	MIN and MAX with operands of mixed sign, MINU, MAXU, NEG, NAND, XNOR and CMP
tb_NbitPR	8	Pass-through on the rising edge, holding the value while `STL` is high over two cycles, and resuming once the stall is released
tb_NbitRegister	8	Loading when enabled, holding when the write enable is low, and clearing
tb_ProgramCounter	6	CLR, counting with CE, JMP, JAL, a JR returning to the saved address, and JMP taking priority over CE

Table 14: VHDL testbenches used to verify individual components.

Test benches were not written for the control unit, the shift unit or the forward select unit, nor for the complete CPU, as a test bench for the full design would have required every block diagram to be converted to VHDL and every pipeline stage to be modelled by hand. Instead, these parts of the design were verified on the board using the method described in the following sections.

5.2 On-Board Debugging

To observe the CPU while it runs, the clock can be switched from the 50 MHz board clock to a divided clock of approximately 1 Hz, which is slow enough for the state of the pipeline to be read after each rising edge. A number of internal signals were connected to the LEDs of the board so that the control signals asserted for the instruction in the decode stage can be seen directly, as listed in Table 18. The push buttons select which internal value is shown on the fifth and sixth seven segment displays (Table 19). These include the opcode in the decode stage, the program counter, and a code identifying which source the execute output funnel has selected, which is decode using Table 17. Comparing these values against what each instruction of a program should produce allowed a fault to be narrowed down to a particular stage, and often to a particular control signal. This method was used throughout the integration of the CPU and caught several errors that would have been difficult to locate otherwise. For example Pipeline Registers not updating, `PC` not updating, and the `JR` jumping back to the `JAL` instruction causing infinite loop, among other minor mistakes. The block implementation is shown in Figure 20.

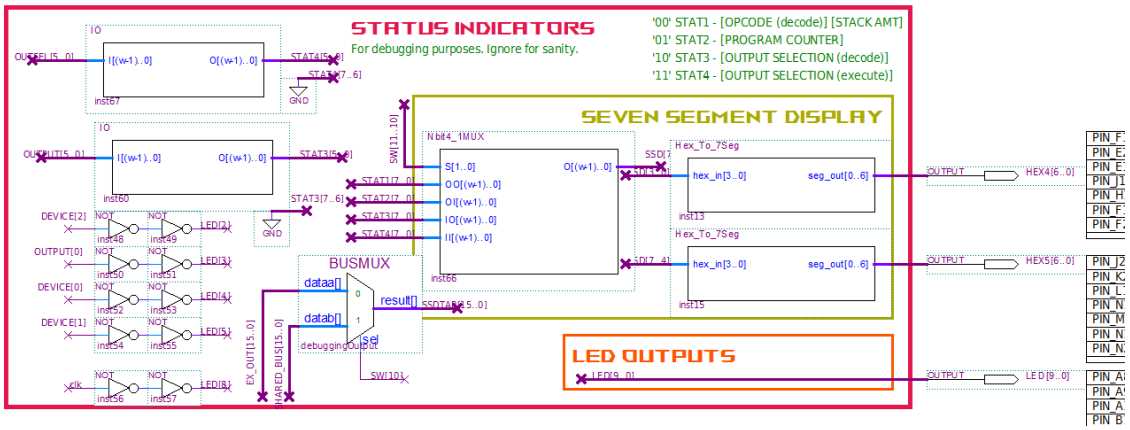


Figure 20: Block Diagram of Status Indicators on FPGA used for debugging purposes.

5.3 Self-Checking Test Programs

Since inspecting the indicators by hand can be slow, a set of test programs was written in assembly that check their own results (Appendix A). The first, `echo`, is a minimal program that copies the switches to the seven segment display in a loop and serves only to confirm, that fetching, the IO instructions and a backwards jump all work. The two self-testing programs then excercise the remainder of the instruction set. `selftest_alu` performs 29 checks covering the register file, every ALU1, ALU2 and shift function in both register and immediate form where available, and stores and loads to the data memory. `selftest_flow` performs 34 checks covering every branch condition, both taken and not taken with signed and unsigned operands, as well as JMP, JAL and JR and NOP. The `selftest_alu.asm` helped discover ALU2 Register Write error within the control unit where a mistake "`if not funct ='111' then`", register write enable, which obviously doesn't work as the correct intended "`if not (funct = '111') then`", since this wasn't used in our `calc.asm` program it was not caught until proper testing was done.

Each check runs a single instruction with known operands, compares the result against the expected value with a CMP, and branches to a failure routine if they differ. Before each check the program loads a unique failure code into register with LDI, so that a faulty ALU instruction cannot corrupt the code it is about to report. When a check fails the CPU displays this code, `EOnn` for `selftest_alu` or `E1nn` for `selftest_flow`, where `nn` identifies the failed test, and then halts. If every check passes the display shows `600D`. The programs are intended to be run in the order `echo`, `selftest_alu`, then `selftest_flow`, since each one relies on the instructions verified by the previous program. Every check in `selftest_alu` relies on CMP and BNE, while `selftest_flow` assumes that LDI, ADD, SUB and CMP are already working.

Finally, the `calc` program, which implements a four-function calculator on the switches, was used as a demonstration that the CPU can run a complete program combining subroutine calls, loops and IO.

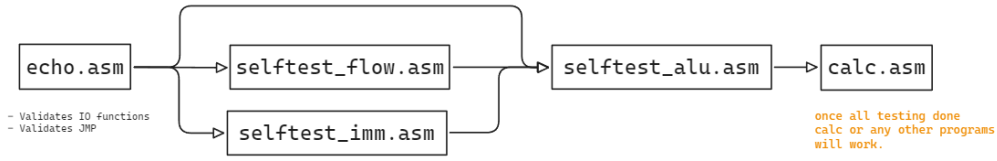


Figure 21: Program dependencies and debugging flow used in cpu desing.

5.4 Results

From the testing done, all tests were passed outputting to the seven segment display "600D"–this indicates all tests passing, and all code appear functional. Issues arose with our hardware debugging control bits and status indicators, switching clock signals between 50MHz and 1Hz quickly could be causing timing errors but this has not properly been diagnosed. Furthermore, the Quartus compilations show ***timing requirements not met*** error but this has not been seen to affect operation of the code ran, examples of this are shown below, however solving them is likely beyond the scope of this assignment.

```
// example snippet...
//Critical Warning (332148): Timing requirements not met
//Info (332146): Worst-case setup slack is -11.557
//        Info (332119):     Slack       End Point TNS Clock
//        Info (332119): ========= =================== =====================
//        Info (332119):   -11.557          -36685.649 ADC_CLK_10
//        Info (332119):    -7.423             -39.860 NbitPR:DeviceSelRegister|Q[0]
//        Info (332119):    -2.223             -34.081 MAX10_CLK2_50
//Info (332146): Worst-case hold slack is -0.452
//        Info (332119):     Slack       End Point TNS Clock
//        Info (332119): ========= =================== =====================
```

```
//        Info (332119):    -0.452              -0.452 ADC_CLK_10
//        Info (332119):    -0.028              -0.028 NbitPR:DeviceSelRegister|Q[0]
//        Info (332119):     0.312               0.000 MAX10_CLK2_50
```

Program	Checks	Display	Result	
echo	–	Switch value	Pass. Display follows the switches	
selftest_alu	29	600D	Pass. All ALU1, ALU2, shift and memory checks	
selfttest_flow	34	600D	Pass. All branch conditions, JMP, JAL, JAR and NOP	
selftest_imm	22	600D	Pass. Immediate forms of ALU2 and CMP	
calc	–	Result, A, B	Pass. Expected results for all four operations	

Table 15: Results of the test programs on the final design.

Fault	Found By	Cause	Fix
ALU2 results not writen to registers	selftest_alu, T15 (E00F)	the condition `if not funct = "111"` is evaluated as `(not funct)="111"`, so the register write was only enabled for MIN	Changed to `if funct /="111"` and added `funct` to the process sensitivity list
MAXU returned the signed maximum	selftest_alu, T20 (E014)	MAXU compared the operands with `signed()`. The testbench used two positie operands, where signed and unsigned comparisons agree, so it did not detect the fault	Changed the comparison to `unsigned()`
SRA had logical error	selftest_alu, T24 (E024)	A shift for $amt > 1$ length was performed, however the bits for negative values were not filled for the full length	Changed the Shif_right(unsigned(x), amt..) to shift_right(signed(x), amt...) which handles it fully, and removed previous incorrect supporting logic.

Table 16: Some Faults found during testing and their fixes

6 Discussion

The design aimed to deliver a working 16-bit pipe-lined CPU that had no pipeline bubbles, rather than trying to maximise clock frequency– meaning it won't run as quickly. A few noteable decisions helped shape the overall architecture and final result.

Firstly, the memory access and branch resolution was chosen to be placed in the Decode stage. The Decode stage is traditionally strict with containing only the identifying instruction type and reading registers and the memory access and branch resolution are handled in later stages–to many functional units and paths create lots of transistor gating which necessitates slower clock speed to allow for the full traversal within the pipelines clock cycle. This choice means that the data RAM can be read while the instruction is decoded and the value is ready when the instruction reaches the execute stage. Similarly, the branch unit outputs JMP, JAL, and JR directly into the program counter. Both of these decisions remove stall and the branch penalty that appear in conventional CPU designs, all of this possible without any stall logic (STL inputs of every pipeline register aren't used). In future work an exploration of pipeline rtl designs for higher clock frequencies can be explored.

Secondly, the design has the aforementioned synchronous ROM output to the IF/ID register and returns two words per fetch (Dout and DoutI). This allows for the design to have 32-bit instructions with a 16-bit immediate. It also keeps the chosen Harvard architecture simple, since the instructions and data never share a bus.

Finally, the register file, other registers, and most edge triggered memory components that are not involved in the pipeline process, were chosen to be written on the falling edge of the clock. While the pipeline registers are registered on the rising edge. this decision allows the results in the stages to be stable before the next rising edge. Likewise, the flag latches are designed so the flags of a CMP reach the branch unit within the same cycle - this is unconventional for regular FPGA design and usually add lots of timing complications, however, our tests reported no issues with the implementation.

A few limitations were prevalent in the CPU design. One such, was the memory addresses being fixed within the instruction, which is what made the aforementioned Decode-stage memory access possible, but it prevented the addition of pointers, arrays, and register-indirect addressing. Another limitation was the architecture size limiting program size and register pressure resulting in multiplication being required to be done on software. Finally, a quality of life limitation was that most system testing and verification relies on the hardware producing a status LEDs as opposed to having an automated test bench.

Future improvements to the design must be taken into consideration as well. Firstly, the instruction set could be condensed with JMP, JAL, and JR sharing one opcode with two function bits, and NOP and HALT could share another. This would free up opcodes for new instructions. Another improvement would be altering the level-sensitive flag latch to become and edge-triggered flag register. An another notable improvement would be designing a self-checking VHDL test bench which would be automated by assembly - this would be easier if the entire design was VHDL instead of a mix as the group ran into a few issues as a result of this.

7 Conclusion

This project produced a working 16-bit pipe-lined CPU. The design widens the philosophy of the 8-bit reference CPU provided by Ben Eater transforming it into a 16-bit Harvard architecture CPU, adds a 16-instruction ISA with both a 16 and 32 bit format, and divides exection into Fetch, Decode, and Execute stages with a write-back phase. The ALU, ALU2, shifter, control unit, ROM, RAM, and register file are written in VHDL while the PC, forwarding, and pipeline were schematic designs in Quartus. The decode-stage branching with memory access and a forwarding unit allowed the pipeline within the design to run without stalls or bubbles which was a target when the CPU was being designed. The project helped develop the groups skills in VHDL, Block Diagrams, CPU Desing, Testing and Verification, and demonstrated how architectural choices have drastic impacts on the final design. The research conducted in this project will allow for entry exploration into the more advanced designs prevalant within the industry today.

References

[1] eddiewastaken. “Reference 8-bit cpu design. ”[Online]. Available: `https://github.com/eddiewastaken/logisim-discrete-CPU`

A Assembly of Code ran on the CPU

A.1 Assembly code for ’echo’ program

```
# ============================================================================
# echo.asm - bring-up test: switches -> display
# ============================================================================
# The smallest possible "is the CPU alive" program. Reads the switches and
# shows the value on the seven-segment display, forever.
#
# Expected: the display always equals the switch value (e.g. all switches
# down -> 0000, SW0 up -> 0001, SW3 and SW0 up -> 0009).
#
# Exercises: SWI, SSD, JMP (backwards).
# ============================================================================
```

```
LOOP:
SWI R1
SSD R1
JMP LOOP
```

A.2 Assembly Code for Calc Program

*Note this is before the addition an Immediate version for ALU2, so some of these instructions may be redundant.

```
# ============================================================================
# calc.asm -- interactive switch calculator: + - x /
# ============================================================================
# Reads the switches on every pass, performs one operation and shows the
# result on the seven-segment display, forever. Flip switches while it runs
# and the display follows.
#
# SWITCH LAYOUT (value returned by SWI)
#   SW[3:0]    A     0..15
#   SW[7:4]    B     0..15
#   SW[9:8]    OP    0 = A + B
#                    1 = A - B   (shown as an 8-bit two's complement byte)
#                    2 = A x B   (shift-and-add subroutine)
#                    3 = A / B   (repeated subtraction subroutine)
#   SW[15:10]  ignored
#
# DISPLAY LAYOUT   [ Out ][ A ][ B ]  = four hex digits  O O A B
#   Out is one byte: the sum, difference or product; for a division it is
#   [remainder][quotient], one digit each. A divide by zero shows BADB.
#
# THINGS TO TRY (A, B, OP -> display)
#    3,  4, 0 -> 0734          3,  4, 1 -> FF34          3,  4, 2 -> 0C34
#   15, 15, 0 -> 1EFF         15, 15, 2 -> E1FF
#   15,  4, 3 -> 33F4  (15 / 4 = 3 remainder 3)         7, 2, 3 -> 1372
#    5,  3, 3 -> 2153  (5 / 3 = 1 remainder 2)          9, 0, 3 -> BADB
#
# REGISTERS
#   R1  raw switch value, later the RESULT
#   R2  temporary                  R3  A          R4  B
#   R5  OP, later a working copy   R6  working copy
#   R7  link register for JAL/JR (assumed; nothing here keeps data in R7)
#   R0  not used
#   A and B (R3, R4) are never modified after they are decoded, so they can
#   be shown next to the result.
#
# Exercises: SWI, SSD, LDI, ADD/SUB/AND/OR (reg and imm), INC, SLL, SRL, CMP
#            (reg and imm), BEQ, BLTU, JMP, JAL, JR.
# ============================================================================

MAIN:
SWI R1                      # R1 = raw switches
AND R3 R1 0x0F              # R3 = A = SW[3:0]
SRL R4 R1 4                 # R4 = SW >> 4
AND R4 R4 0x0F              # R4 = B = SW[7:4]
```

```
SRL R5 R1 8                 # R5 = SW >> 8
AND R5 R5 0x03              # R5 = OP = SW[9:8]

# ---- choose the operation: OP 0, 1, 2, else 3 ----
CMP R5 0
BEQ DO_ADD
CMP R5 1
BEQ DO_SUB
CMP R5 2
BEQ DO_MUL
JMP DO_DIV

DO_ADD:
ADD R1 R3 R4                # result = A + B
JMP SHOW

DO_SUB:
SUB R1 R3 R4                # result = A - B
JMP SHOW

DO_MUL:
JAL MULTIPLY                # result = A x B
JMP SHOW

DO_DIV:
JAL DIVIDE                  # result = [remainder][quotient], or BADB

SHOW:
CMP R1 0xBADB
BEQ SHOW_RAW                # divide by zero: show BADB as it is
SLL R1 R1 8                 # Out -> [15:8]
SLL R2 R3 4                 # A   -> [7:4]
OR  R1 R1 R2                # [Out][A][0]    (back-to-back dependent instructions)
OR  R1 R1 R4                # [Out][A][B]
SHOW_RAW:
SSD R1
JMP MAIN

# ---- subroutine: R1 = R3 x R4 by shift-and-add; keeps R3 and R4 ----
MULTIPLY:
LDI R1 0                    # accumulator
OR  R5 R3 R3                # R5 = multiplicand (copy of A)
OR  R6 R4 R4                # R6 = multiplier   (copy of B)
MUL_LOOP:
CMP R6 0                    # multiplier used up?
BEQ MUL_DONE
AND R2 R6 1                 # lowest bit of the multiplier
CMP R2 0
BEQ MUL_SKIP                # bit clear -> nothing to add
ADD R1 R1 R5                # accumulator += multiplicand
MUL_SKIP:
SLL R5 R5 1                 # multiplicand *= 2
SRL R6 R6 1                 # multiplier /= 2
JMP MUL_LOOP
MUL_DONE:
```

```
JR

# ---- subroutine: R1 = [remainder][quotient] of R3 / R4; keeps R3 and R4 ----
DIVIDE:
CMP R4 0                    # B = 0?
BEQ DIV_ZERO
LDI R1 0                    # quotient
OR  R6 R3 R3                # R6 = what is left of A (becomes the remainder)
DIV_LOOP:
CMP R6 R4                   # what is left < B ?
BLTU DIV_DONE               # yes: finished
SUB R6 R6 R4                # take B out of it ...
INC R1 R1                   # ... and count it
JMP DIV_LOOP
DIV_DONE:
SLL R6 R6 4                 # remainder -> [7:4]
OR  R1 R1 R6                # [remainder][quotient]
JR
DIV_ZERO:
LDI R1 0xBADB
JR
```

B Testing. Decoding Tables

EXECUTE OUPUT CONTROL DEFINITIONS	
OUTPUT CODE	**Description**
01	IO is on either Switch to execute output or Seven Segment Register In (Check LEDR2)
02	A Register (rs1) pass through to execute ouptut
04	Immediate to execute output
08	Shift Unit to execute output
10	ALU2 Unit to execute output
20	Ram Memory to execute output
40	

Table 17: Code used to decode the funnel output in execute stage of the pipeline.

LED ID	Signal Description
0	Stack is Full
1	Stack is Empty
2	Seven Segment Display Register is on Write Enable
3	SWO is enabled (Switch input to execute output)
4	Register is being written to
5	RAM is being written to
6	the slow clock
7	Count is enabled (Either by +2 or +1)
8	Clock Signal in
9	If on Program is halted.

Table 18: Table used to decode CPU functionality with LEDs.

Push Button	Description
00	Displays the OPCODE in decode and Amt of Items in the stack
01	Displays the Program Counter value (Address for ROM)
10	Displays the EXECUTE output CODE (Refer to table above 17)
11	

Table 19: Table for selecting output (with push button switches) onto seven segment display 5 and 6.

C Assembler

C.1 Code Examples

A generic Assembler was created. The design process was iterative so having a generic assembler aided in making quick changes to the ISA, and therefore its definition in the Assembler. Below is an example of an ISA definition within the Assembler class.

```
public class Assembler {
    // ================================================================================
    // 1. CONFIGURATION
    // ================================================================================
    /** true = comma-separated operands, false = whitespace-separated. */
    static final boolean USE_CSV_INPUT = false;
    static final int INSTRUCTION_WIDTH = 16;   // bits per word
    static final int OPCODE_WIDTH = 4;
    static final int HEX_DIGITS = (INSTRUCTION_WIDTH + 3) / 4;
    /** Shown in the "source" column for the second word of a 32-bit instruction. */
    static final String IMMEDIATE_WORD_LABEL = "(immediate)";

    // ================================================================================
    // 2. FUNCTION / CONDITION TABLES  ({mnemonic, bit pattern})
    // ================================================================================
    // Every name here becomes a mnemonic (see buildIsa). These must be
    // declared above ISA_DEFINITION, which is built during class
    // initialization and reads them.
    static final String[][] ALU = { // These take 2 inputs and can be defined together for ALU1
        {"ADD", "000"},
        {"SUB", "001"},
        {"AND", "100"},
        {"OR",  "101"},
        {"XOR", "110"},
        };
    static final String[][] ALU1IN = { // These take 1 input define interpret seperately for ALU1
        {"INC", "010"},
        {"DEC", "011"},
        {"NOT", "111"}};

    static final String[][] ALU2 = {
        {"MIN",  "000"},
        {"MAX",  "001"},
        {"MINU", "010"},
        {"MAXU", "011"},
        {"NAND", "101"},
        {"XNOR", "110"}};

    static final String[][] ALU21IN = {
```

```
    {"NEG",  "100"}};

/** Compare has its own table because it takes no destination register. */
static final String[][] CMP = {
    {"CMP", "111"}};

static final String[][] SHIFT = {
    {"SLL",  "00"},
    {"SRL", "01"},
    {"SRA",  "10"},
    {"ROL", "11"}};
static final String[][] IO = {
    {"SSD", "000"},
    {"SWI", "001"}};
static final String[][] BR = {
    {"BEQ",  "000"},
    {"BNE",  "001"},
    {"BLT",  "010"},
    {"BGE",  "011"},
    {"BLE",  "100"},
    {"BGT",  "101"},
    {"BLTU", "110"},
    {"BGEU", "111"}};
// ============================================================================
// 3. ISA DEFINITION
// ============================================================================
// An instruction layout is an opcode plus a list of fields. The list order
// is both the order of operands in the source and, within each word, the
// order of bits from the most-significant bit downward. Every word must
// total INSTRUCTION_WIDTH bits (word 1 includes the opcode).
//
// Field kinds:
//     reg(n)          register such as R3
//     imm(n)          immediate value (decimal or hex)
//     immNextWord()   16-bit immediate stored in a second word
//                     (makes the instruction two words long)
//     addr(n)         numeric address or label
//     amt(n)          small unsigned number such as a shift amount (0..2^n-1)
//     func(n)         function bits, supplied by the mnemonic (ADD, SUB, ...);
//                     no operand in the source
//     cond(n)         same as func(n); reads better for branch conditions
//     nullField(n)    padding that still appears in the source as NULL or 0
//     unused(n)       unused bits: no operand in the source, encoded as 0
//
// Two ways to register mnemonics:
//     define(...)         one mnemonic, one layout            (LW, JMP, HALT, ...)
//     defineFamily(...)   every name in a table becomes a mnemonic that uses
//                         the given layout(s). If a family has several
//                         layouts, the assembler picks the one whose
//                         operands fit (ADD,R1,R2,R3 vs ADD,R1,R2,10).
static final Map<String, List<Form>> ISA_DEFINITION = buildIsa();

static Map<String, List<Form>> buildIsa() {
    Map<String, List<Form>> isa = new LinkedHashMap<>();
```

```
        define(isa, "NOP", instr("0000", unused(12)));                    // NOP             e.g.  NO
        //ALU
        defineFamily(isa, ALU,      instr("0001", reg(3), reg(3), reg(3), func(3)));          // ADD,
        defineFamily(isa, ALU1IN,      instr("0001", reg(3), reg(3), unused(3), func(3)));
        //ALUI
        defineFamily(isa, ALU,     instr("0010", reg(3), reg(3),unused(3), func(3),imm16()); // ADD,Rd,

        define(isa, "LW",  instr("0011", reg(3), addr(9)));          // LW    Rd, address
        define(isa, "SW",  instr("0100", reg(3), addr(9)));          // SW    Rs, address
        define(isa, "LDI", instr("0101", reg(3), unused(9), imm16()));// LDI   Rd, immediate
        defineFamily(isa, BR,       instr("0111", cond(3), addr(9)));          // BEQ,address
        define(isa, "JMP", instr("1000", unused(3),addr(9)));        // JMP   address
        define(isa, "JAL", instr("1001", unused(3),addr(9)));        // JAL   address
        define(isa, "JR",  instr("1010", unused(12)));                        // JR
        defineFamily(isa, SHIFT,    instr("1011", reg(3), reg(3), amt(4), func(2)));    // SL,Rd,Rs,amt
        // ALU2
        defineFamily(isa, ALU2,     instr("1100", reg(3), reg(3), reg(3), func(3)));    // MIN,Rd,Rs1,Rs
        defineFamily(isa, ALU21IN,     instr("1100", reg(3), reg(3), unused(3), func(3)));
        defineFamily(isa, CMP,      instr("1100", unused(3),reg(3), reg(3), func(3)); // CMP,Rs1,Rs2
        // ALU2I
        defineFamily(isa, ALU2,     instr("1101", reg(3), reg(3), unused(3), func(3), imm16()));    // I
        defineFamily(isa, CMP,      instr("1101", unused(3),reg(3), unused(3), func(3), imm16())); // CI

        defineFamily(isa, IO,       instr("1110", reg(3), unused(6), func(3)));               // SSD,Rs

        define(isa, "HALT", instr("1111", unused(12)));// HALT                          e.g.  HALT
        return isa;
    }
}
```

Below is the function that converts the interpreted Assembly code, if successful into the 'binaries' or rather a form that can be simply copy and pasted into the ROM definition that hold the ISA.

```
static void writeOutput(List<AssembledLine> program,Map<String, Integer> labels, String outputPath) thro
    boolean csv = outputPath.toLowerCase().endsWith(".csv");
    StringBuilder out = new StringBuilder();

    if (csv) {
        out.append("address,hex,binary,source\n");
    }
    for (AssembledLine line : program) {
        if (csv) {
            out.append(toCsvRow(line));
        } else {
            labels.entrySet().stream()
                .filter(entry -> Objects.equals(entry.getValue(), line.address))
                .map(Map.Entry::getKey)
                .findFirst()
                .ifPresent(
                    label -> {out.append("--" + label + ":\n");}
                );
            out.append(String.format("%d => x\"%s\", -- %s", line.address,toHex(line.encoded), line.sour
        }
    }
    Files.write(Paths.get(outputPath), out.toString().getBytes());
}
```